

sqmSQLTool

Complete Function Reference

129 Functions | 22 Categories

Version 2.0 | 2026-06-12

14. Active Directory Integration

Get-sqmADAccountStatus

Checks the status of an Active Directory user account.

Parameter	Description
<code>-SamAccountName</code>	The SamAccountName of the AD account to check.
<code>-DomainController</code>	Optional target DC. Only used via the RSAT path.

Example:

```
Get-sqmADAccountStatus -SamAccountName 'jdoe'
```

Get-sqmADGroupMembers

Lists all members of an Active Directory group.

Parameter	Description
<code>-GroupName</code>	Name of the AD group. Pipeline-capable. Format: "GroupName" or "DOMAIN\GroupName"
<code>-Domain</code>	Optional: AD domain (e.g., "FITS.LOCAL", "corp.de") If not specified, auto-detects curr...

Example:

```
Get-sqmADGroupMembers -GroupName "DL_SQL_Admins"
```

Get-sqmHpuAllowGroup

Searches for the HPU allow group in Active Directory based on configurable domain/group mappings.

Example:

```
Get-sqmHpuAllowGroup
```

Remove-sqmAdOrphanLogin

Removes Windows logins whose Active Directory account no longer exists (AD orphans).

Parameter	Description
-SqlInstance	Target SQL Server instance. Default: current computer name.
-SqlCredential	Optional PSCredential for the instance.
-ExcludeLogin	Additional logins to exclude from removal (wildcards allowed). Combined with the always...
-AdModuleAction	Behavior when the ActiveDirectory module is not present: 'Abort' (default) - stop with ...
-BackupPath	Directory for the rollback script. Default: C:\System\WinSrvLog\MSSQL (created if missi...
-SkipBackup	Skip writing the rollback script. Not recommended.
-EnableException	Throw exceptions immediately instead of returning error status.

Example:

```
Remove-sqmAdOrphanLogin -SqlInstance "SQL01" -WhatIf Lists exactly which AD-orphaned logins would be removed,  
without changing anything.
```

10. Reporting & Analysis

Export-sqmDatabaseDocumentation

Creates structured HTML and CSV documentation for all databases on a SQL Server instance.

Parameter	Description
<code>-SqlInstance</code>	SQL Server instance(s). Pipeline-capable. Default: current computer name.
<code>-SqlCredential</code>	Optional PSCredential for the SQL connection.
<code>-Database</code>	Document specific databases only. Wildcards allowed (e.g. 'Sales*'). Default: all user...
<code>-IncludeSystemDatabases</code>	Include system databases (master, model, msdb, tempdb). Default: \$false.
<code>-IncludeFileDetails</code>	Include filegroup and file details in the report. Default: \$true.
<code>-IncludeUsers</code>	Include database users in the report. Default: \$true.
<code>-IncludeObjectSummary</code>	Include object summary (tables, SPs, views, etc.) in the report. Default: \$true.
<code>-OutputPath</code>	Output directory. Default: value from module configuration (Get-sqmDefaultOutputPath).
<code>-ContinueOnError</code>	Continue on error for an instance or database instead of aborting.
<code>-EnableException</code>	Throw exceptions directly (overrides ContinueOnError).
<code>-Confirm</code>	Request confirmation before writing output files.
<code>-WhatIf</code>	Simulation: shows which files would be created without writing them.

Example:

```
Export-sqmDatabaseDocumentation
```

Get-sqmAutoGrowthReport

Creates an AutoGrowth configuration report for all database files on a SQL Server instance.

Parameter	Description
-SqlInstance	SQL Server instance (default: current computer name).
-SqlCredential	PSCredential for the connection.
-Database	Restrict to specific databases (array of names).
-IncludeSystem	Include system databases. Default: \$false.
-Detailed	When set, additional file properties (physical path) are included in the output.
-EnableException	Throw exceptions immediately.

Example:

```
Get-sqmAutoGrowthReport -SqlInstance "SQL01"
```

Get-sqmServerHardwareReport

Erstellt einen HTML-Hardware-Konfigurationsbericht fuer einen oder mehrere Server.

Parameter	Description
-ComputerName	Zielservers (ein oder mehrere). Standard: lokaler Computer. Aliase: SqlInstance, ServerName
-ReportPath	Ausgabepfad fuer die HTML-Report-Datei(en). Standard: %ProgramData%\sqmSQLTool\Hardware...
-OutputFormat	Ausgabeformat: HTML (Standard), CSV, TXT oder All (alle Formate gleichzeitig). - HTML: ...
-NoOpen	HTML-Datei nach dem Erstellen NICHT automatisch im Browser oeffnen.
-PassThru	Gibt den vollstaendigen Pfad der erstellten Datei(en) als String zurueck.
-EnableException	Ausnahmen sofort ausloesen statt Write-Error.

Example:

Invoke-sqmInstanceInventory

Creates a complete inventory of a SQL Server instance as a structured report (TXT + CSV).

Parameter	Description
-SqlInstance	SQL Server instance(s). Pipeline-capable. Default: current computer name.
-SqlCredential	Optional PSCredential for the connection.
-OutputPath	Output directory for report files. Default: value from module configuration (Get-sqmDe...
-ContinueOnError	Continue on error for an instance (otherwise abort).
-EnableException	Allow exceptions to pass through (for advanced error handling).
-Confirm	Request confirmation before creation.
-WhatIf	Test only, do not write files.

Example:

```
Invoke-sqmInstanceInventory
```

Invoke-sqmSetupReport

Professional SQL Server Setup Report with critical issues, security, and database overview.

Parameter	Description
-SqlInstance	SQL Server instance. Default: local computer name.
-SqlCredential	Credentials for SQL connection.
-OutputPath	Output path for HTML report.
-PassThru	Return the file path.
-NoOpen	Don't open the report in browser.

Parameter	Description
-----------	-------------

Example:

```

Invoke-sqmSetupReport -SqlInstance "SQL01" #> function Invoke-sqmSetupReport { [CmdletBinding()]
[OutputType([string])] param ( [Parameter(Mandatory = $false, Position = 0)] [string]$SqlInstance,
[Parameter(Mandatory = $false)] [System.Management.Automation.PSCredential]$SqlCredential, [Parameter(Mandatory =
$false)] [string]$OutputPath, [Parameter(Mandatory = $false)] [switch]$PassThru, [Parameter(Mandatory = $false)]
[switch]$NoOpen ) begin { $functionName = $MyInvocation.MyCommand.Name if (-not
$PSBoundParameters.ContainsKey('SqlInstance') -or [string]::IsNullOrEmpty($SqlInstance)) { $SqlInstance =
$env:COMPUTERNAME } if (-not $PSBoundParameters.ContainsKey('OutputPath') -or
[string]::IsNullOrEmpty($OutputPath)) { $OutputPath = Get-sqmConfig -Key 'OutputPath' if (-not $OutputPath)
{ $OutputPath = "C:\System\WinSrvLog\MSSQL" } } Invoke-sqmLogging -Message "Starte $functionName auf
$SqlInstance" -FunctionName $functionName -Level 'INFO' } process { try { $timestamp = Get-Date -Format 'yyyy-MM-
dd HH:mm:ss' $safeInstance = $SqlInstance -replace '[\:\]', ' ' $datestamp = Get-Date -Format 'yyyyMMdd HHmm'
$server = $null # Connect to SQL Server try { $server = Connect-DbInstance -SqlInstance $SqlInstance
-SqlCredential $SqlCredential -ErrorAction Stop } catch { throw "Verbindung zu $SqlInstance fehlgeschlagen: $
($_.Exception.Message)" } # ===== # CRITICAL ISSUES #
===== # SA Account Status $saLogin = Get-DbLogin -SqlInstance $server
-Login 'sa' -ErrorAction SilentlyContinue if (-not $saLogin) { $saSid = '0x01' $saLogin = Get-DbLogin
-SqlInstance $server | Where-Object { $_.SID -eq $saSid } } $saName = if ($saLogin) { $saLogin.Name } else { 'NOT
FOUND' } $saDisabled = if ($saLogin) { $saLogin.IsDisabled } else { $null } $saIsRenamed = ($saName -ne 'sa')
$saStatus = if ($saIsRenamed -and $saDisabled) { 'OK (renamed & disabled)' } elseif ($saIsRenamed) { 'OK
(renamed)' } elseif ($saDisabled) { 'WARNING (disabled only)' } else { 'CRITICAL (not renamed)' } $saStatusColor
= if ($saName -ne 'sa' -or $saDisabled) { 'green' } else { 'red' } # Backup Jobs Status $backupJobs = Get-
DbAgentJob -SqlInstance $server -ErrorAction SilentlyContinue | Where-Object { $_.Name -like '*backup*' -or
$.Name -like '*bkp*' } $backupJobCount = @($backupJobs).Count $backupJobsEnabled = @($backupJobs | Where-Object
{ $_.IsEnabled }).Count $backupJobStatus = if ($backupJobCount -eq 0) { 'NO BACKUP JOBS' } elseif
($backupJobsEnabled -eq $backupJobCount) { "OK ($backupJobCount jobs)" } else { "WARNING
($backupJobsEnabled/$backupJobCount enabled)" } $backupStatusColor = if ($backupJobCount -gt 0 -and
$backupJobsEnabled -eq $backupJobCount) { 'green' } else { 'orange' } # Max Memory (synchronized with Test-
sqmMaxMemory logic) $maxMem = $server.Configuration.MaxServerMemory.ConfigValue $totalMem =
[math]::Round($server.PhysicalMemory / 1024, 0) $unconfiguredValue = 2147483647 $lowerBound =
[math]::Round($totalMem * 0.85) $upperBound = [math]::Round($totalMem * 0.95) if ($maxMem -eq $unconfiguredValue)
{ $maxMemStatus = "NOT CONFIGURED (default)" $maxMemColor = 'orange' } elseif ($maxMem -gt $upperBound)
{ $maxMemStatus = "TOO HIGH ($maxMem MB > $upperBound MB)" $maxMemColor = 'orange' } elseif ($maxMem -lt
$lowerBound) { $maxMemStatus = "TOO LOW ($maxMem MB < $lowerBound MB)" $maxMemColor = 'orange' } else
{ $maxMemStatus = "OK ($maxMem MB)" $maxMemColor = 'green' } # ===== #
SECURITY CHECKS # ===== # Sysadmin Accounts $sysadmins = @() try
{ $sysadmins = @(Get-DbLogin -SqlInstance $server | Where-Object { $_.IsSysAdmin -eq $true } | Select-Object
-ExpandProperty Name) } catch { } $sysadminList = if ($sysadmins.Count -gt 0) { $sysadmins -join ', ' } else
{ 'None' } # Logins with Server Roles (only server-level roles) $advancedLogins = @() try { $serverRoles =
@('sysadmin', 'serveradmin', 'securityadmin', 'processadmin', 'dbcreator', 'diskadmin') $allLogins = Get-DbLogin
-SqlInstance $server -ErrorAction SilentlyContinue foreach ($login in $allLogins) { $roles = @() foreach ($role
in $serverRoles) { try { $query = "SELECT IS_SRVROLEMEMBER('$role', '$($login.Name)') AS IsMember" $result =
Invoke-DbQuery -SqlInstance $server -Query $query -ErrorAction SilentlyContinue if ($result -and
$result.IsMember -eq 1) { $roles += $role } } catch { } } if ($roles.Count -gt 0) { $advancedLogins += "$
($login.Name) [$($roles -join ', ')]" } } catch { } $advancedLoginList = if ($advancedLogins.Count -gt 0)
{ $advancedLogins -join ' | ' } else { 'None with server roles' } # CLR Status $clrEnabled =
$server.Configuration.IsSqlClrEnabled.ConfigValue $clrStatus = if ($clrEnabled) { 'ENABLED (check if needed)' }
else { 'OK (disabled)' } $clrColor = if ($clrEnabled) { 'orange' } else { 'green' } # xp_cmdshell Status
$xpCmdEnabled = $server.Configuration.XPCmdShell.ConfigValue $xpStatus = if ($xpCmdEnabled) { 'ENABLED (security
risk)' } else { 'OK (disabled)' } $xpColor = if ($xpCmdEnabled) { 'orange' } else { 'green' } #
===== # SERVICE ACCOUNTS & INFRASTRUCTURE #
===== # Service Accounts $serviceAccounts = @() try { # SQL Server Service
$sqlSvc = Get-Service -ErrorAction SilentlyContinue | Where-Object { $_.Name -match "MSSQL\SQL Server" -and
$.Name -notmatch "Agent|Browser" } | Select-Object -First 1 if ($sqlSvc) { $svcInfo = Get-CimInstance -ClassName
Win32_Service -Filter "Name='$($sqlSvc.Name)'" -ErrorAction SilentlyContinue if ($svcInfo) { $serviceAccounts +=
"SQL Server: $($svcInfo.StartName)" } } # SQL Agent Service $agentSvc = Get-Service -ErrorAction SilentlyContinue
| Where-Object { $_.Name -match "SQLSERVERAGENT\SQLAgent" } | Select-Object -First 1 if ($agentSvc) { $svcInfo =
Get-CimInstance -ClassName Win32_Service -Filter "Name='$($agentSvc.Name)'" -ErrorAction SilentlyContinue if
($svcInfo) { $serviceAccounts += "SQL Agent: $($svcInfo.StartName)" } } } catch { } $serviceAccountList = if
($serviceAccounts.Count -gt 0) { $serviceAccounts -join ' | ' } else { 'Unable to determine' } # SPN Status (List
all SPNs) $spnList = 'Not checked' $spnDetails = @() try { $spnReport = Get-sqmSpnReport -ComputerName
$env:COMPUTERNAME -ErrorAction SilentlyContinue if ($spnReport -and $spnReport.DetailRows) { foreach ($row in
$spnReport.DetailRows) { $spnDetails += "$($row.SPN) [$($row.Status)]" } } $spnList = if ($spnDetails.Count -gt 0)
{ $spnDetails -join ' | ' } else { 'No SPNs found' } } catch { $spnList = 'Error retrieving SPNs' } # Splunk
Status (via Invoke-sqmSplunkConfiguration -Test) $splunkStatus = 'Not configured' try { $splunkResult = Invoke-
sqmSplunkConfiguration -Mode Test -ErrorAction SilentlyContinue if ($splunkResult) { if
($splunkResult.IsConfigured) { $splunkStatus = "Configured (service: $($splunkResult.ServiceStatus))" } else
{ $splunkStatus = 'Not configured' } } } catch { $splunkStatus = 'Error checking Splunk' } #
===== # CONFIGURATION # ===== # MAXDOP
$maxdop = $server.Configuration.MaxDegreeOfParallelism.ConfigValue $cpuCount = $server.Processors
$recommendedMaxdop = if ($cpuCount -le 4) { $cpuCount } elseif ($cpuCount -le 8) { 4 } elseif ($cpuCount -le 16)
{ 8 } else { 16 } $maxdopStatus = if ($maxdop -ge 2 -and $maxdop -le $recommendedMaxdop) { "OK ($maxdop)" } else

```

```
{ "CHECK ($maxdop, recommended $recommendedMaxdop)" } # Cost Threshold $ctp =
$server.Configuration.CostThresholdForParallelism.ConfigValue $ctpStatus = if ($ctp -ge 50) { "OK ($ctp)" } else
{ "WARNING ($ctp, recommended >= 50)" } # TempDB $tempdb = Get-DbaDatabase -SqlInstance $server -Database
'tempdb' $tempdbFileCount = $tempdb.FileGroups.Files.Count $idealCount = [Math]::Min($cpuCount, 8) $tempdbStatus
= if ($tempdbFileCount -eq $idealCount) { "OK ($tempdbFileCount files)" } else { "CHECK ($tempdbFileCount files,
ideal $idealCount)" } # ===== # DATABASES #
===== $databases = @() try { $allDbs = Get-DbaDatabase -SqlInstance $server
-ExcludeSystem foreach ($db in $allDbs) { $dbo = $db.Owner $lastBackup = $db.LastFullBackupDate $daysAgo = if
($lastBackup) { (New-TimeSpan -Start $lastBackup -End (Get-Date)).Days } else { -1 } $backupStatus = if ($daysAgo
-lt 0) { 'Never' } elseif ($daysAgo -eq 0) { 'Today' } elseif ($daysAgo -le 7) { "$daysAgo days" } else
{ "$daysAgo days △" } $databases += [PSCustomObject]@{ Name = $db.Name Recovery = $db.RecoveryModel DBO = $dbo
LastFullBackup = $backupStatus } } catch { } # ===== # BUILD HTML #
===== $html = Build-ModernReportHtml ` -SqlInstance $SqlInstance `
-Timestamp $timestamp ` -SAStatus $saStatus ` -SAStatusColor $saStatusColor ` -SASName $saName ` -BackupStatus
$backupJobStatus ` -BackupStatusColor $backupStatusColor ` -MaxMemStatus $maxMemStatus ` -MaxMemColor
$maxMemColor ` -Sysadmins $sysadminList ` -AdvancedLogins $advancedLoginList ` -CLRStatus $clrStatus ` -CLRColor
$clrColor ` -XPStatus $xpStatus ` -XPColor $xpColor ` -ServiceAccounts $serviceAccountList ` -SPNList $spnList `
-SplunkStatus $splunkStatus ` -MAXDOP $maxdopStatus ` -CostThreshold $ctpStatus ` -TempDB $tempdbStatus `
-Databases $databases # ===== # SAVE REPORT #
===== if (-not (Test-Path $OutputPath)) { $null = New-Item -ItemType
Directory -Path $OutputPath -Force } $htmlFile = Join-Path $OutputPath "sgmSetupReport ${safeInstance} $
{datestamp}.html" $html | Out-File -FilePath $htmlFile -Encoding UTF8 -Force Invoke-sgmOpenReport -HtmlFile
$htmlFile -NoOpen:$NoOpen Invoke-sgmLogging -Message "Report erstellt: $htmlFile" -FunctionName $functionName
-Level 'INFO' Write-Host "✅ Setup-Report: $htmlFile" -ForegroundColor Green Copy-sgmToCentralPath -Path
$htmlFile if ($PassThru) { return $htmlFile } } catch { $errMsg = "Fehler: $($.Exception.Message)" Invoke-
sgmLogging -Message $errMsg -FunctionName $functionName -Level 'ERROR' Write-Error $errMsg } } #
===== # HTML Builder Function #
===== function Build-ModernReportHtml
{ param( [string]$SqlInstance, [string]$Timestamp, [string]$SAStatus, [string]$SAStatusColor, [string]$SASName,
[string]$BackupStatus, [string]$BackupStatusColor, [string]$MaxMemStatus, [string]$MaxMemColor, [string]
$Sysadmins, [string]$AdvancedLogins, [string]$CLRStatus, [string]$CLRColor, [string]$XPStatus, [string]$XPColor,
[string]$ServiceAccounts, [string]$SPNList, [string]$SplunkStatus, [string]$MAXDOP, [string]$CostThreshold,
[string]$TempDB, [PSCustomObject[]]$Databases ) function HtmlEncode { param([string]$Text) if (-not $Text)
{ return '' } $Text -replace '&', '&amp;' -replace '<', '&lt;' -replace '>', '&gt;' -replace '"', '&quot;'
-replace "'", '&#39;' } $dbRows = if ($Databases) { $Databases | ForEach-Object { "<tr><td>$( HtmlEncode
$.Name)</td><td>$( $.Recovery)</td><td>$( HtmlEncode $.DBO)</td><td>$( $.LastFullBackup)</td></tr>" } | Out-
String } else { "<tr><td colspan='4'>No databases</td></tr>" } return @"<!DOCTYPE html> <html lang="de"> <head>
<meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>SQL Server
Setup Report - $( HtmlEncode $SqlInstance)</title> <style> * { box-sizing: border-box; margin: 0; padding: 0; }
body { font-family: 'Segoe UI', Arial, sans-serif; background: #060f20; color: #e2e8f0; font-size: 14px; line-
height: 1.6; }
```

3. Backup & Recovery

Get-sgmOperationStatus

Displays progress and estimated remaining time for active backup, restore and AutoSeed operations.

Parameter	Description
-SqlInstance	The target SQL Server instance (default: current computer name).
-SqlCredential	Alternative credentials.
-OperationType	Filters by operation type. Valid values: 'Backup', 'Restore', 'AutoSeed'. By default a...

Parameter	Description
-Continuous	When set, output is continuously refreshed (similar to 'watch'). Stop with Ctrl+C.
-RefreshSeconds	Refresh interval in seconds for continuous mode (default: 5). Only used with -Continuous.
-EnableException	Switch to allow exceptions to pass through (by default errors are logged as warnings).

Example:

```
# Show all active operations on the local instance Get-sqmOperationStatus
```

Invoke-sqmRestoreDatabase

Restores a database from a backup file, with support for single-server and AlwaysOn environments.

Parameter	Description
-SqlInstance	Target SQL Server instance (e.g. "localhost", "SQL01\INSTANCE"). Default: current compu...
-SqlCredential	Alternative credentials for the target instance.
-BackupFile	Path to the full backup file (.bak). Can also be an array for striped backups. For seq...
-BackupFiles	Array of backup files in order: Full, then Diff (optional), then Logs (optional). Exam...
-DatabaseName	Name of the database to restore (as it appears in the backup file). Required to determi...
-NewDatabaseName	Optional: New name for the database after the restore. If specified, logical file names...
-NewDatabaseFilePath	Optional: Target directory for database files (.mdf, .ndf). If not

Parameter	Description
	specified, the defau...
-NewLogFilePath	Optional: Target directory for the log file (.ldf). If not specified, the default direc...
-BackupBeforeRestore	Optional: Creates a full backup of the existing database before the restore (if present...
-NoUserExport	Optional: Skips export of database users (users are always exported by default). The e...
-KeepAlwaysOn	Optional: If the database is part of an AG, it is not removed from the AG. Note: Resto...
-WithNoRecovery	Optional: Performs the restore with NORECOVERY so the database remains in restoring sta...

Example:

```
# Simple restore of a full backup file Invoke-sqlmRestoreDatabase -SqlInstance "SQL01" -BackupFile "D:\Backup\AdventureWorks.bak" -DatabaseName "AdventureWorks"
```

Invoke-sqlmUserDatabaseBackup

Backs up user databases on a SQL Server instance.

Parameter	Description
-SqlInstance	The target SQL Server instance (e.g. "localhost", "SQL01\INSTANCE"). If not specified, ...
-SqlCredential	Alternative credentials (PSCredential). If not specified, Windows authentication is used.
-Database	Name or array of user databases to back up. Ignored when -All is set.
-All	When set, all user databases on the instance are backed up.

Parameter	Description
<code>-BackupPath</code>	Optional direct backup path (overrides the value from server properties). The path must...
<code>-UseExcludeTable</code>	When set, reads master.dbo.sqm_BackupExclude and skips databases where IsActive=1 and I...
<code>-CheckPreferredReplica</code>	When set, checks sys.fn_hadr_backup_is_preferred_replica() for all AG databases on this...
<code>-MailTo</code>	Recipient email address for the backup report. When specified, a mail is sent via SQL S...
<code>-MailProfile</code>	SQL Server Database Mail profile name to use for sending the report mail. Default: 'Def...
<code>-MailOnSuccess</code>	When set together with -MailTo, a report mail is also sent when all backups succeeded (...)
<code>-EnableException</code>	Switch to propagate exceptions immediately (by default errors are logged as warnings).

Example:

```
# Back up all user databases on the current computer Invoke-sqmUserDatabaseBackup -All
```

New-sqmBackupMaintenanceJob

Creates a SQL Agent job with two steps that implement the full dynamic backup maintenance workflow.

Parameter	Description
<code>-SqlInstance</code>	SQL Server instance. Default: current computer name (\$env:COMPUTERNAME).
<code>-SqlCredential</code>	PSCredential for the SQL connection.

Parameter	Description
-JobName	Name of the SQL Agent job to create. Default: 'sqm-BackupMaintenance-FULL'.
-BackupType	Backup type: 'FULL', 'DIFF', or 'LOG'. Default: 'FULL'.
-BackupPath	Optional backup path. When specified, overrides the server default and is passed as -B...
-ScheduleTime	Start time of the schedule in format 'HH:mm'. Default: '20:00'.
-ScheduleDays	Days of the week for the schedule. Valid values: 'Monday'..'Sunday', 'Weekdays', 'Weeke...
-ScheduleIntervalMinutes	Repeat interval within a day in minutes (e.g. 15 = every 15 minutes). 0 = run once at ...
-JobCategory	SQL Agent job category. Default: 'Database Maintenance'.
-UseExcludeTable	When set, passes -UseExcludeTable to Invoke-sqmUserDatabaseBackup in Step 2.
-CheckPreferredReplica	When set, passes -CheckPreferredReplica to Invoke-sqmUserDatabaseBackup in Step 2.
-IncludeSystemDatabases	When set, passes -IncludeSystemDatabases to Sync-sqmBackupExcludeTable in Step 1. Note...

Example:

```
# Weekly FULL backup Sunday 20:00 with all features New-sqmBackupMaintenanceJob -SqlInstance "SQL01" -BackupType FULL -UseExcludeTable -CheckPreferredReplica -MailTo "dba@company.com" -MailProfile "DBA-Mail"
```

Set-sqmBackupExcludePermission

Grants SELECT, INSERT, and UPDATE permissions on master.dbo.sqm_BackupExclude to a login.

Parameter	Description
-SqlInstance	The target SQL Server instance (e.g. "localhost", "SQL01\INSTANCE"). If not specified, ...
-SqlCredential	Alternative credentials (PSCredential). If not specified, Windows authentication is used.
-LoginName	The Windows group (e.g. "DOMAIN\DBA-Team") or SQL login to grant permissions to. This p...
-EnableException	Switch to propagate exceptions immediately (by default errors are logged as warnings).

Example:

```
# Grant permissions to a Windows group on the local instance Set-sqmBackupExcludePermission -LoginName "CONTOSO\DBA-Team"
```

Sync-sqmBackupExcludeTable

Creates and synchronises the backup exclude table in the master database.

Parameter	Description
-SqlInstance	The target SQL Server instance (e.g. "localhost", "SQL01\INSTANCE"). If not specified, ...
-SqlCredential	Alternative credentials (PSCredential). If not specified, Windows authentication is used.
-IncludeSystemDatabases	When set, the system databases master, model, and msdb are also inserted into the exclu...
-EnableException	Switch to propagate exceptions immediately (by default errors are logged as warnings).

Example:

```
# Synchronise on the local instance - user databases only Sync-sqmBackupExcludeTable
```

Test-sqmBackupIntegrity

Verifies one or more backup files using RESTORE VERIFYONLY.

Parameter	Description
<code>-SqlInstance</code>	SQL Server instance on which the verification runs (default: current computer name).
<code>-SqlCredential</code>	PSCredential for the connection.
<code>-BackupPath</code>	Path to the backup file (.bak) on the server (local path, not UNC). Can be an array. I...
<code>-FileListOnly</code>	When \$true, only lists the files contained in the backup (without VerifyOnly).
<code>-EnableException</code>	Throw exceptions immediately.

Example:

```
Test-sqmBackupIntegrity -SqlInstance "SQL01" -BackupPath "D:\Backup\AdventureWorks.bak"
```

7. Configuration Management

Compare-sqmServerConfiguration

Compares important configuration settings between two SQL Server instances.

Parameter	Description
<code>-SourceInstance</code>	Source instance (reference).
<code>-TargetInstance</code>	Target instance (server to compare). Mandatory.
<code>-SqlCredential</code>	PSCredential for both instances (if identical). For different credentials, separate par...
<code>-CompareDatabases</code>	When set, databases (Name, Owner, RecoveryModel, Collation) are

Parameter	Description
	compared.
-EnableException	Throw exceptions immediately.
Example: <pre>Compare-sqmServerConfiguration -SourceInstance "SQL01" -TargetInstance "SQL02"</pre>	
Export-sqmServerConfiguration	
Exports all SQL Server configuration settings to a JSON snapshot file.	
Parameter	Description
-SqlInstance	Target SQL Server instance (default: \$env:COMPUTERNAME).
-SqlCredential	Optional alternative credentials (PSCredential object).
-OutputPath	Path where JSON snapshot will be saved. Default: \$env:ProgramData\sqmSQLTool\Snapshots
-Label	Optional descriptive label for this snapshot (e.g., "before MaxMemory change"). Include...
-IncludeDatabases	When set, includes database-level settings (slower operation).
-EnableException	Switch to allow exceptions to pass through (default: errors logged as warnings).

Example:

```
# Create a snapshot before making configuration changes $snap = Export-sqmServerConfiguration -SqlInstance "SQL01" -Label "before MaxMemory change" Write-Host "Snapshot saved to: $($snap.SnapshotPath)"
```

Invoke-sqmConfigRollback

Restores SQL Server configuration from a previously exported snapshot.

Parameter	Description
-SqlInstance	Target SQL Server instance (default: \$env:COMPUTERNAME).

Parameter	Description
-SqlCredential	Optional alternative credentials (PSCredential object).
-SnapshotPath	Full path to the JSON snapshot file to restore from. Required parameter.
-Category	Limit rollback to specific categories. Valid values: 'SpConfigure', 'InstanceProperties...
-WhatIf	Show what would be changed without making actual modifications.
-Force	Skip confirmation dialog and apply changes immediately.
-EnableException	Switch to allow exceptions to pass through (default: errors logged as warnings).

Example:

```
# Preview what would be restored Invoke-sqmConfigRollback -SqlInstance "SQL01" -SnapshotPath "C:\Snapshots\SQL01_MSSQLSERVER_20260602_143022.json" -WhatIf
```

Set-sqmConfig

Sets one or more configuration values for the MSSQLTools module.

Parameter	Description
-LogPath	Directory for log files (Invoke-sqmLogging).
-OutputPath	Default output directory for reports.
-CentralPath	Optional central storage directory (additional copy).
-OlaJobNameFull	Name of the full backup job for user databases.
-OlaJobNameDiff	Name of the diff backup job for user databases.
-OlaJobNameLog	Name of the log backup job for user databases.

Parameter	Description
<code>-OlaJobNameIndexOpt</code>	Name of the IndexOptimize job.
<code>-OlaJobNameIntUserDb</code>	Name of the IntegrityCheck job for user databases.
<code>-OlaJobNameIntSysDb</code>	Name of the IntegrityCheck job for system databases.
<code>-OlaJobNameSysDbBackup</code>	Name of the full backup job for system databases.
<code>-TsmManagementClasses</code>	Array of valid TSM management classes (e.g. 'MC_B_NL.NL_42.42.NA').
<code>-HpuDomainGroupMap</code>	Array of PSCustomObject with fields DomainPattern (wildcard) and GroupNamePattern (sAM...

Example:

```
Set-sqmConfig -LogPath "D:\Logs" -OlaJobNameFull "Prod-FULL"
```

Get-sqmConfig

Returns the current module configuration.

Parameter Description

`-Key` Name of the configuration key (e.g. 'LogPath', 'OutputPath', 'CentralPath').

Example:

```
Get-sqmConfig
```

Set-sqmTcpPort

Konfiguriert den TCP-Port einer SQL Server-Instanz ueber die Registry.

Parameter	Description
<code>-SqlInstance</code>	Name der SQL-Instanz (z.B. "MSSQLSERVER" fuer Default, "INST01" fuer Named Instance). D...
<code>-BasePort</code>	Basisport. Standard: 1433.

Parameter	Description
-PortIncrement	Schrittweite pro Instanz. Standard: 10.
-InstanceNumber	Optionale explizite Instanznummer (ueberschreibt Auto-Erkennung aus Instanzname).

Example:

```
Set-sqmTcpPort -SqlInstance 'MSSQLSERVER' -BasePort 1433
```

Invoke-sqmCollationChange

Automatically changes the server collation of a SQL Server instance.

Parameter	Description
-SqlInstance	SQL Server instance (must be local). Default: current computer name.
-SqlCredential	PSCredential for the SQL connection.
-NewCollation	Target collation (e.g. 'Latin1_General_CI_AS').
-IncludeUserDatabases	When set, the default collation of all user databases is also changed.
-BackupBeforeChange	Creates a full backup of all user databases before the change.
-ExcludeDatabase	Databases to exclude from -IncludeUserDatabases (wildcards allowed).
-ServiceName	Windows service name (automatically determined from SqlInstance if not specified).
-StartupTimeoutSeconds	Maximum wait time for sqlservr.exe in minimal mode (default: 120).
-OutputPath	Output directory for rollback documentation and column script.

Parameter	Description
	Default: Get-sqmDefault...
-ContinueOnError	Continue with the next step on error (rarely used).
-EnableException	Throw exceptions immediately.
-Confirm	Request confirmation before stopping the service and making the change.

Example:

```
Invoke-sqmCollationChange -NewCollation "Latin1_General_CI_AS"
```

17. SSRS Configuration

Set-sqmSrsHttpsCertificate

Bind a Windows certificate to SSRS or Power BI Report Server for HTTPS access.

Parameter	Description
-ComputerName	Target computer name or IP address. Default: localhost (\$env:COMPUTERNAME).
-Thumbprint	Mandatory. Certificate thumbprint (40 hex characters) from the LocalMachine\My store. ...
-Port	HTTPS port to bind. Default: 443.
-InstanceName	SSRS WMI instance name (e.g. "RS_SSRS", "RS_PBIRS"). Auto-detected when only one inst...
-IPAddress	IP address for the SSL binding. Default: "0.0.0.0" (all interfaces).
-RequireSSL	When specified, sets SecureConnectionLevel = 1 (HTTPS required). Default: SecureConne...

Parameter	Description
-Credential	PSCredential for the WinRM session (remote operation only).
-WhatIf	Shows what would happen without making any changes.
-Confirm	Prompts for confirmation before applying changes.

Example:

```
Set-sqmSrsHttpsCertificate -Thumbprint "A1B2C3D4E5F6A1B2C3D4E5F6A1B2C3D4E5F6A1B2" Binds the specified certificate to SSRS on the local machine on port 443. The SSRS instance is auto-detected.
```

5. Login & User Security

Copy-sqmLogins

Copies logins from a source SQL Server instance to a target instance.

Parameter	Description
-Source	Source SQL Server instance. Mandatory.
-Destination	Target SQL Server instance. Mandatory.
-SqlCredential	Optional PSCredential for both instances (source and target). For different credential...
-SourceCredential	PSCredential specifically for the source instance.
-DestinationCredential	PSCredential specifically for the target instance.
-Login	Filters the copy operation to these login names (wildcards allowed). Without specifica...
-ExcludeLogin	Logins that should not be copied (wildcards allowed). Example: 'AppLogin_*', 'OldUser'.
-IncludeSystemLogins	When set, system logins (sa, ##MS_*, NT SERVICE*, NT

Parameter	Description
	AUTHORITY*) are also copied. De...
-DisablePolicy	Controls whether the default policy on the target is disabled before copying and re-en...
-AdjustAuthMode	When set and the target is Windows-only auth but the source uses Mixed Mode, the targe...
-RestartServiceIfRequired	When set, the SQL Server service on the target server is automatically restarted after...
-Force	Existing logins on the target server are overwritten.

Example:

```
Copy-sqmLogins -Source 'SQL01' -Destination 'SQL02' Copies all non-system logins. Policy is disabled/re-enabled, AD check and orphan repair run automatically.
```

Get-sqmADAccountStatus

Checks the status of an Active Directory user account.

Parameter	Description
-SamAccountName	The SamAccountName of the AD account to check.
-DomainController	Optional target DC. Only used via the RSAT path.

Example:

```
Get-sqmADAccountStatus -SamAccountName 'jdoe'
```

Get-sqmADGroupMembers

Lists all members of an Active Directory group.

Parameter	Description
-GroupName	Name of the AD group. Pipeline-capable. Format: "GroupName" or "DOMAIN\GroupName"

Parameter	Description
-----------	-------------

-Domain	Optional: AD domain (e.g., "FITS.LOCAL", "corp.de") If not specified, auto-detects curr...
----------------	--

Example:

```
Get-sqmADGroupMembers -GroupName "DL SQL Admins"
```

Get-sqmLoginSettings

Zeigt alle Logins mit Default-Datenbank und Spracheinstellung.

Parameter	Description
-----------	-------------

-SqlInstance	SQL Server-Instanz(en). Pipeline-faehig. Standard: aktueller Computernamen.
---------------------	---

-SqlCredential	Optionales PSCredential.
-----------------------	--------------------------

-LoginType	Filter: 'All' (Standard), 'SQL', 'Windows'
-------------------	--

-ExcludeSystemLogins	NT SERVICE*, NT AUTHORITY*, ##MS_*** automatisch ausblenden.
-----------------------------	--

-DefaultDatabase	Filter: nur Logins mit dieser Default-Datenbank anzeigen.
-------------------------	---

-DefaultLanguage	Filter: nur Logins mit dieser Sprache anzeigen.
-------------------------	---

-OutputPath	Wenn angegeben, wird eine CSV-Datei geschrieben. Standard: kein Export.
--------------------	---

-ContinueOnError	Bei Fehler auf einer Instanz fortfahren.
-------------------------	--

-EnableException	Fehler sofort als Ausnahme auslösen.
-------------------------	--------------------------------------

Example:

```
Get-sqmLoginSettings
```

Get-sqmSysadminAccounts

Retrieves all logins with sysadmin rights on a SQL Server instance.

Parameter	Description
<code>-SqlInstance</code>	SQL Server instance(s). Pipeline-capable. Default: current computer name.
<code>-SqlCredential</code>	Optional PSCredential for the connection.
<code>-ExcludeLogin</code>	Logins to be marked as 'Excluded' (wildcards allowed).
<code>-ExcludeSysAccounts</code>	When set, known system accounts are automatically excluded.
<code>-IncludeDisabled</code>	If \$true (default), disabled sysadmin logins are also included.
<code>-OutputPath</code>	Output directory for report files. Default: C:\System\WinSrvLog\MSSQL
<code>-ContinueOnError</code>	Continue on error for an instance.
<code>-EnableException</code>	Throw exceptions immediately (overrides ContinueOnError).
<code>-Confirm</code>	Request confirmation before writing files.
<code>-WhatIf</code>	Shows which files would be created without writing them.

Example:

```
Get-sqmSysadminAccounts
```

Invoke-sqmSaObfuscation

Obfuscates the SA account on a SQL Server instance by renaming it, disabling it, and setting a random password.

Parameter	Description
<code>-SqlInstance</code>	Target SQL Server instance(s). Pipeline-capable. Default: current computer name.
<code>-SqlCredential</code>	Optional PSCredential for the SQL connection.

Parameter	Description
-NewName	New name for the SA account. Default: 'sqmsa'.
-PasswordLength	Length of the random password (12-128). Default: 18.
-ContinueOnError	Continue with the next instance on error (otherwise aborts).
-EnableException	Throw exceptions immediately (overrides ContinueOnError).
-Confirm	Prompts for confirmation before critical changes (default: off).
-WhatIf	Shows what would happen without making any changes.

Example:

```
Invoke-sqmSaObfuscation -SqlInstance "SQL01"
```

Remove-sqmAdOrphanLogin

Removes Windows logins whose Active Directory account no longer exists (AD orphans).

Parameter	Description
-SqlInstance	Target SQL Server instance. Default: current computer name.
-SqlCredential	Optional PSCredential for the instance.
-ExcludeLogin	Additional logins to exclude from removal (wildcards allowed). Combined with the always...
-AdModuleAction	Behavior when the ActiveDirectory module is not present: 'Abort' (default) - stop with ...
-BackupPath	Directory for the rollback script. Default: C:\System\WinSrvLog\MSSQL (created if missi...
-SkipBackup	Skip writing the rollback script. Not recommended.

Parameter	Description
-----------	-------------

-EnableException Throw exceptions immediately instead of returning error status.

Example:

```
Remove-sqmAdOrphanLogin -SqlInstance "SQL01" -WhatIf Lists exactly which AD-orphaned logins would be removed, without changing anything.
```

Set-sqmDatabaseOwner

Sets the owner of one or more databases to a uniform login.

Parameter	Description
-SqlInstance	SQL Server instance(s). Pipeline-capable. Default: current computer name.
-SqlCredential	PSCredential for the connection.
-Database	Database name(s). Wildcards allowed (e.g. 'Prod*'). Default: all user databases.
-ExcludeDatabase	Databases to exclude. Wildcards allowed.
-OwnerLogin	Login to set as the new owner. Default: sa account (automatically determined via SID 0...
-IncludeSystemDatabases	Also include system databases (master, model, msdb). Default: \$false. tempdb is always...
-Force	Also process databases that already have the correct owner (forces re-assignment).
-OutputPath	Directory for the change log. Default: from module configuration.
-ContinueOnError	Continue on error for one instance. Default: \$false.
-EnableException	Throw exceptions immediately.

Example:

```
# Set sa account on all user databases Set-sqmDatabaseOwner -SqlInstance "SQL01"
```

12. Server Configuration Testing

Get-sqmClusterInfo

Retrieves information about a Windows Failover Cluster: cluster name, nodes and roles including IP addresses.

Parameter	Description
-ClusterName	The name of the cluster to query. If not specified, the function attempts to determine ...
-IncludeCoreGroup	Switch to include the core cluster group ("Cluster Group") in the roles list. Storage ...
-NoAutoInstall	Suppresses the automatic installation of RSAT clustering tools if the module is missing.
-EnableException	When set, errors are thrown as exceptions (by default an error object is returned).

Example:

```
$info = Get-sqmClusterInfo -ClusterName "MYCLUSTER" if (-not $info.Success) { Write-Error $info.ErrorMessage;  
return } $info.ClusterName $info.Nodes | Format-Table $info.Roles | Where-Object OwnerNode -eq "Node1" | Select  
Name, IPAddresses
```

Get-sqmPerfCounters

Reads SQL Server performance counters from sys.dm_os_performance_counters.

Parameter	Description
-SqlInstance	SQL Server instance. Default: local computer name.
-SqlCredential	PSCredential for the connection.
-Category	Filter on category fragments, e.g. @('Buffer','Memory','Locks'). Default: all

Parameter	Description
	key count...
-TopN	Maximum number of results. Default: 50.
-OutputPath	If specified, a CSV report is saved.
-EnableException	Throw exceptions immediately.

Example:

```
Get-sqmPerfCounters -SqlInstance "SQL01"
```

Get-sqmSpnReport

Checks the registered SPNs for SQL Server instances (default and named instances).

Parameter	Description
-ComputerName	Target computer. Default: local computer. Pipeline-capable.
-InstanceFilter	Optional filter on instance names (wildcards allowed). Example: 'MSSQLSERVER' for defau...
-OutputPath	Output directory for report and CSV. Default: module configuration (Get-sqmConfig -Key ...
-ContinueOnError	Continue with the next instance on error.
-EnableException	Throw exceptions immediately (overrides ContinueOnError).
-Confirm	Request confirmation before creating files.
-WhatIf	Shows which files would be created without writing them.

Example:

```
Get-sqmSpnReport Checks all SQL Server instances on the local computer.
```

Test-sqmSQLFirewall

Tests whether the firewall and network allow a TCP connection to SQL Server.

Parameter	Description
-Server	Hostname or IP address of the SQL Server. Pipeline-capable.
-Port	TCP port to test. Default: 1433. Ignored when -Instance is specified and the SQL Browse...
-Instance	Name of the named instance (without server prefix). When specified, the SQL Browser (UD...
-TimeoutSeconds	Timeout for the TCP connection test in seconds. Default: 5.
-ContinueOnError	Continue with the next server on error instead of aborting.
-EnableException	Throw exceptions immediately (overrides ContinueOnError).

Example:

```
Test-sqmSQLFirewall -Server "SQL01" Tests the default instance on TCP port 1433.
```

Test-sqmSqlInstanceInstalled

Prueft ob eine SQL Server-Instanz auf dem lokalen System installiert ist.

Parameter	Description
-InstanceName	Name der zu pruefenden SQL-Instanz. Default: "MSSQLSERVER" (Default-Instanz).

Example:

```
Test-sqmSqlInstanceInstalled # Prueft Default-Instanz MSSQLSERVER
```

4. Monitoring & Health Checks

Get-sqmBlockingReport

Retrieves current blocking chains on a SQL Server instance.

Parameter	Description
-SqlInstance	SQL Server instance (default: current computer name).

Parameter	Description
-SqlCredential	PSCredential for the connection.
-MinWaitSeconds	Only report blocking that has been waiting longer than this value (in seconds). Default...
-OutputPath	If specified, a CSV snapshot is written to this directory.
-EnableException	Throw exceptions immediately instead of returning as errors.

Example:

```
Get-sqmBlockingReport
```

Get-sqmConnectionStats

Analyzes active SQL Server connections and connection statistics.

Parameter	Description
-SqlInstance	SQL Server instance. Default: local computer name.
-SqlCredential	PSCredential for the connection.
-GroupBy	Grouping criterion: Application Login Host Database. Default: Application.
-TopN	Number of top groups. Default: 25.
-IncludeSystemConnections	Include system connections (is_user_process = 0).
-OutputPath	If specified, a CSV report is saved.
-EnableException	Throw exceptions immediately.

Example:

```
Get-sqmConnectionStats -SqlInstance "SQL01"
```

Get-sqmDatabaseHealth

Aggregated health report for all databases on an instance.

Parameter	Description
<code>-SqlInstance</code>	SQL Server instance(s). Pipeline-capable. Default: current computer name.
<code>-SqlCredential</code>	Optional PSCredential for the connection.
<code>-MaxCheckDbAgeDays</code>	Maximum age in days of the last error-free DBCC CHECKDB. Default: 14.
<code>-MaxVlfCount</code>	Warning threshold for VLF count per database. Default: 200.
<code>-HistoryDays</code>	Time range for AutoGrowth evaluation in days. Default: 30.
<code>-ExcludeDatabase</code>	Databases to exclude. Wildcards allowed.
<code>-IncludeSystemDatabases</code>	Include system databases (except tempdb). Default: \$false.
<code>-OutputPath</code>	Output directory for report files. Default: C:\System\WinSrvLog\MSSQL
<code>-ContinueOnError</code>	Continue on error for an instance (otherwise the error is thrown).
<code>-EnableException</code>	Throw exceptions immediately (overrides ContinueOnError).
<code>-Confirm</code>	Request confirmation before writing files.
<code>-WhatIf</code>	Shows which files would be created without actually writing them.

Example:

```
Get-sqmDatabaseHealth
```

Get-sqmDeadlockReport

Reads and analyzes deadlock events from the System Health Extended Event session.

Parameter	Description
-SqlInstance	SQL Server instance (default: current computer name).
-SqlCredential	PSCredential for the connection.
-StartTime	Return only deadlocks from this point in time. Default: last 24 hours.
-EndTime	Return only deadlocks up to this point in time. Default: now.
-MaxDeadlocks	Maximum number of deadlocks returned (newest first). Default: 100.
-OutputPath	If specified, deadlock graphs are saved as .xdl files in this directory (format: Deadl...
-EnableException	Throw exceptions immediately instead of returning as errors.

Example:

```
Get-sqmDeadlockReport
```

Get-sqmLongRunningQueries

Identifies long-running queries on a SQL Server instance.

Parameter	Description
-SqlInstance	SQL Server instance (default: current computer name).
-SqlCredential	PSCredential for the connection.
-MinDurationSeconds	Return only queries running longer than this value (seconds). Default: 30.
-MinCpuMs	Return only queries whose CPU time exceeds this value (milliseconds). Default: 0.
-ExcludeWaitType	Wait types to exclude (e.g. 'SLEEP_TASK','WAITFOR'). Default:

Parameter	Description
	common idle waits.
-IncludeSystemSessions	Include system sessions (SPID <= 50) as well. Default: \$false.
-IncludeQueryPlan	Retrieve the XML execution plan as well (expensive - only on demand). Default: \$false.
-OutputPath	If specified, a CSV snapshot is written to this directory.
-EnableException	Throw exceptions immediately instead of returning as errors.

Example:

```
Get-sqmLongRunningQueries
```

Get-sqmServerSetting

Reads one or all server properties from a SQL Server instance.

Parameter	Description
-SqlInstance	The target SQL Server instance (e.g. "localhost", "SQL01\INSTANCE"). If not specified,...
-SqlCredential	Alternative credentials (PSCredential). If not specified, Windows Authentication is used.
-Name	The name of the server property to retrieve. Only the following values are allowed: Ba...
-All	When set, all properties of the server object are returned as a list.
-DefaultValue	Optional default value if the property does not exist or cannot be read. Ignored when ...
-EnableException	Switch to allow exceptions to pass through (by default errors are logged as warnings).

Example:

```
# Read BackupDirectory from the local server $backupPath = Get-sqmServerSetting -Name "BackupDirectory"
```

Get-sqmSQLInstanceCheck

Checks a SQL Server instance against best practices.

Parameter	Description
-SqlInstance	The target SQL Server instance (default: current computer name).
-SqlCredential	Alternative credentials.
-Detailed	Detailed output (e.g. path checks, analyze all databases). Default: \$false.
-EnableException	Allow exceptions to pass through.

Example:

```
Get-sqmSQLInstanceCheck
```

Invoke-sqmLoginAudit

Comprehensive audit of all SQL Server logins on one or more instances.

Parameter	Description
-SqlInstance	SQL Server instance(s). Pipeline-capable. Default: current computer name.
-SqlCredential	Optional PSCredential.
-InactivityThresholdDays	Logins without login since this value are considered inactive. Default: 90.
-MaxPasswordAgeDays	SQL logins with password older than this value are reported (non-sysadmins). Default: 1...
-MaxPasswordAgeDaysSysadmin	SQL logins with password older than this value are reported (sysadmins). Default: 365. ...
-ExcludeLogin	Logins to exclude (wildcards). E.g. 'NT SERVICE*', 'sqmsa'.

Parameter	Description
<code>-IncludeSystemLogins</code>	When set, NT SERVICE*, NT AUTHORITY* are also included.
<code>-CheckPolicyNonSysadmin</code>	Check password policy violations for non-sysadmin logins. Default: \$true.
<code>-CheckPolicySysadmin</code>	Check password policy violations for sysadmin logins. Default: \$true.
<code>-ReportBuiltInAdmins</code>	When BUILTIN\Administrators is found in logins, report as warning. Default: \$true.
<code>-CheckAdOrphans</code>	When set, AD orphan check is performed for Windows logins (requires AD module).
<code>-GenerateHtmlReport</code>	Generate HTML report in addition to TXT/CSV. Default: \$true.

Example:

```
Invoke-sqmLoginAudit
```

8. Storage & Disk Management

Copy-sqmNTFSPermissions

Copies NTFS permissions (ACLs) from a source path to a destination path.

Parameter	Description
<code>-SourcePath</code>	Source path (e.g. "D:\" or "D:\Data").
<code>-DestinationPath</code>	Destination path (e.g. "E:\" or "E:\Data").
<code>-Recurse</code>	Recursive traversal of all subfolders and files.

Parameter	Description
-CreateMissingFolders	Automatically creates missing target folders (directories only, not files). Files missi...
-IncludeSystemAndHidden	Includes hidden and system objects in the processing.

Example:

```
Copy-sqmNTFSPermissions -SourcePath "D:\" -DestinationPath "E:\" -Recurse Copies all permissions from D: to E: (recursively).
```

Get-sqmDiskBlockSize

Prueft die NTFS-Blockgroesse (Cluster-Groesse) von Laufwerken auf 64KB.

Parameter	Description
-Drive	Laufwerkbuchstabe(n) ohne Doppelpunkt, z.B. 'F', 'G', 'H'. Pipeline-faehig. Wenn nicht ...
-SqlInstance	SQL Server-Instanz. Wenn angegeben werden automatisch alle von SQL Server genutzten Lau...
-ComputerName	Zielcomputer fuer die WMI-Abfrage. Standard: lokaler Computer.
-RecommendedBlockSize	Empfohlene Blockgroesse in Bytes. Standard: 65536 (64 KB).
-EnableException	Ausnahmen sofort ausloesen statt Write-Error.

Example:

```
# Einzelne Laufwerke pruefen Get-sqmDiskBlockSize -Drive 'F', 'G', 'H'
```

Get-sqmDiskInfoByDriveLetter

Returns disk information for a given drive letter.

Parameter	Description
-DriveLetter	Drive letter of the volume (e.g. "C", "C:" or "D:").

Parameter	Description
-----------	-------------

-NoClipboard Suppresses copying the result to the clipboard.

Example:

```
Get-sqmDiskInfoByDriveLetter -DriveLetter "C" Returns disk information for drive C: and copies it to the clipboard.
```

Get-sqmOrphanedFiles

Finds MDF/LDF/NDF database files that are not assigned to any database.

Parameter	Description
-----------	-------------

-SqlInstance SQL Server instance. Default: local computer name.

-SqlCredential PScredential for the connection.

-SearchPath Directories to search. Default: all unique directories from sys.master_files + SQL Serv...

-FileExtension File extensions to search for. Default: .mdf, .ldf, .ndf

-Recurse Recursively search subdirectories.

-EnableException Throw exceptions immediately.

Example:

```
Get-sqmOrphanedFiles -SqlInstance "SQL01"
```

11. Module & Update Management

Install-sqmAdModule

Ensures that the ActiveDirectory PowerShell module (RSAT) is installed.

Parameter	Description
<code>-SkipIfPresent</code>	If \$true (default) and the module is already present, \$true is returned immediately wi...
<code>-ContinueOnError</code>	When set, the function returns \$false on failed installation instead of throwing an error.
<code>-EnableException</code>	When set, the function throws an exception on failed installation (overrides ContinueO...
<code>-WhatIf</code>	Shows which installation method would be attempted, without executing it.
<code>-Confirm</code>	Request confirmation before installation.

Example:

```
Install-sqmAdModule Checks whether the AD module is present and installs it if necessary.
```

Test-sqmModuleUpdate

Checks all configured update sources for a newer sqmSQLTool version.

Parameter	Description
<code>-Source</code>	Which sources to check. Valid values: GitHub, PSGallery, UNC, All. Default: All
<code>-RepositoryPath</code>	UNC path for the UNC source check. Default: W:\75084-Datenbanken\MSSQL\DEV\sqmSQLTool
<code>-EnableException</code>	Throw exceptions immediately instead of returning error object.

Example:

```
Test-sqmModuleUpdate
```

Test-sqmUpdateViaGitHub

Checks if a newer version of sqmSQLTool is available on GitHub.

Parameter	Description
-Owner	GitHub repository owner. Default: JankeUwe
-Repository	GitHub repository name. Default: sqmSQLTool
-EnableException	Throw exceptions immediately instead of returning error object.

Example:

```
Test-sqmUpdateViaGitHub
```

Test-sqmUpdateViaPSGallery

Checks if a newer version of sqmSQLTool is available on PowerShell Gallery.

Parameter	Description
-ModuleName	Module name to check. Default: sqmSQLTool
-EnableException	Throw exceptions immediately instead of returning error object.

Example:

```
Test-sqmUpdateViaPSGallery
```

Test-sqmUpdateViaUNC

Checks if a newer version of sqmSQLTool is available on a UNC share.

Parameter	Description
-RepositoryPath	UNC path to the sqmSQLTool repository share. Default: W:\75084-Datenbanken\MSSQL\DEV\sq...
-EnableException	Throw exceptions immediately instead of returning error object.

Example:

```
Test-sqmUpdateViaUNC
```

Update-sqmModule

Updates the sqmSQLTool module from GitHub, PSGallery or a UNC share.

Parameter	Description
-Source	Update source. Valid values: GitHub, PSGallery, UNC. Default: GitHub
-RepositoryPath	UNC path for UNC source. Default: W:\75084-Datenbanken\MSSQL\DEV\sqmSQLTool
-Destination	Installation path for the module. Default: C:\Windows\System32\WindowsPowerShell\v1.0\M...
-Force	Install even if no newer version is available.
-EnableException	Throw exceptions immediately.

Example:

```
Update-sqmModule
```

16. SQL Drivers & Tools Installation

Install-sqmDb2Driver

Installiert den IBM DB2 ODBC/CLI-Treiber.

Parameter	Description
-SourcePath	Pfad zum DB2-Installer oder Verzeichnis mit dem Installer. Z.B.: \srv\Treiber\DB2 oder...

Example:

```
Install-sqmDb2Driver -SourcePath '\\srv\Treiber\DB2'
```

Install-sqmJdbcDriver

Installiert den Microsoft JDBC Driver for SQL Server.

Parameter	Description
-SourcePath	Quellpfad wo der JDBC-Installer oder die .jar liegt. Z.B.: \

Parameter	Description
	\srv\Treiber\JDBC oder C:\D...
-DestinationPath	Zielpfad fuer die .jar-Datei. Standard: C:\Program Files\Microsoft JDBC Driver for SQL ...
-UpdateClassPath	Wenn \$true: CLASSPATH-Systemumgebungsvariable wird um den Zielpfad erweitert. Standard:...

Example:

```
Install-sqmJdbcDriver -SourcePath '\\srv\Treiber\JDBC'
```

Install-sqmOdbcDriver

Installiert den Microsoft ODBC Driver for SQL Server.

Parameter	Description
-SourcePath	Pfad zum ODBC-Installer oder Verzeichnis mit dem Installer. Z.B.: \srv\Treiber\ODBC od...
-DriverName	Optionaler Treibername fuer die Vorab-Pruefung via Test-sqmDriverInstalled. Standard: a...

Example:

```
Install-sqmOdbcDriver -SourcePath '\\srv\Treiber\ODBC'
```

Uninstall-sqmDb2Driver

Deinstalliert den IBM DB2 ODBC/CLI-Treiber.

Example:

```
Uninstall-sqmDb2Driver
```

Uninstall-sqmJdbcDriver

Deinstalliert den Microsoft JDBC Driver for SQL Server.

Parameter	Description
-RemoveClassPath	Entfernt den CLASSPATH-Systemeintrag wenn vorhanden. Standard:

Parameter	Description
-----------	-------------

\$false.

Example:

```
Uninstall-sqmJdbcDriver
```

Uninstall-sqmOdbcDriver

Deinstalliert den Microsoft ODBC Driver for SQL Server.

Parameter	Description
-----------	-------------

-DriverName	Optionaler Treibername fuer gezieltes Matching. Standard: Wildcard 'Microsoft ODBC Driv...
--------------------	--

Example:

```
Uninstall-sqmOdbcDriver
```

19. External Systems Integration

Invoke-sqmSplunkConfiguration

Configures the Splunk Universal Forwarder on SQL Server hosts.

Parameter	Description
-----------	-------------

-Mode	Set - Set environment variables and start/restart SplunkForwarder (default). Test - Che...
--------------	---

-Remote	Remote execution via AD OU search. Combine with -SearchOU.
----------------	--

-SearchOU	Distinguished Name or simple OU name. Default: OUServDatabase.
------------------	--

-ComputerList	Explicit server list: string array or path to a text file (# = comment).
----------------------	--

-Credential	Credentials for AD and remoting.
--------------------	----------------------------------

-LogPath	Directory for log files. Default: sqmSQLTool LogPath configuration.
-----------------	---

Parameter	Description
-----------	-------------

-LogCallback	Optional ScriptBlock for GUI logging.
---------------------	---------------------------------------

Example:

```
Invoke-sqmSplunkConfiguration
```

Test-sqmTsmConnection

Tests the connection to an IBM Spectrum Protect (TSM) server using dsmadm.

Parameter	Description
-----------	-------------

-ComputerName	Target computer on which the connection test is performed. Default: current computer name.
----------------------	--

-DsmadmPath	Full path to dsmadm.exe. Determined automatically from the registry if not specified.
--------------------	---

-UserName	TSM user name (USERID from dsm.opt if not specified).
------------------	---

-Password	TSM password as SecureString (PASSWORD from dsm.opt if not specified).
------------------	--

-ServerName	TSM server address (TCPServeraddress from dsm.opt if not specified).
--------------------	--

-DsmOptPath	Full path to dsm.opt. Determined automatically if not specified.
--------------------	--

-Credential	PSCredential for remote access (WinRM).
--------------------	---

-EnableException	Throw exceptions immediately.
-------------------------	-------------------------------

Example:

```
Test-sqmTsmConnection
```

13. SQL Agent & Proxy Jobs

Get-sqmAgentJobHistory

Displays the execution history of SQL Agent jobs.

Parameter	Description
<code>-SqlInstance</code>	SQL Server instance (default: current computer name).
<code>-SqlCredential</code>	PSCredential for the connection.
<code>-JobName</code>	Name or wildcard pattern (e.g. '*Backup*') to filter jobs.
<code>-Status</code>	'Success', 'Failure', 'Retry' or 'Cancelled'. Default: all.
<code>-Since</code>	Show history from this date onwards. Default: today minus 7 days.
<code>-LastX</code>	Instead of a time range: number of last executions per job (e.g. -LastX 5).
<code>-OutputPath</code>	Export as CSV (optional). If specified, a CSV file is created.
<code>-EnableException</code>	Throw exceptions immediately.

Example:

```
Get-sqmAgentJobHistory
```

New-sqmAgentProxy

Erstellt einen SQL Server Credential und einen SQL Agent Proxy und verbindet beide.

Parameter	Description
<code>-SqlInstance</code>	SQL Server-Instanz. Standard: lokaler Computername.
<code>-SqlCredential</code>	PSCredential fuer die SQL-Verbindung (Windows-Auth wenn nicht angegeben).
<code>-CredentialName</code>	Name des neuen SQL Server Credentials (z.B.

Parameter	Description
	"DOMAIN\ServiceAccount").
-ProxyName	Name des neuen SQL Agent Proxys.
-ProxyDescription	Optionale Beschreibung fuer den Proxy.
-WindowsCredential	Windows-Credential direkt als PSCredential uebergeben (kein Dialog). Wenn nicht angegeben...
-WindowsUserName	Optionaler Windows-Benutzername (DOMAIN\User) zur Vorbestueckung des Get-Credential Dia...
-Subsystem	Subsysteme die dem Proxy zugewiesen werden. Mehrfachauswahl moeglich. Gueltiger Werte: ...
-Force	Ueberschreibt bestehenden Credential und/oder Proxy wenn vorhanden.
-EnableException	Ausnahmen sofort ausloesen statt Write-Error.

Example:

```
# Einzeiler - Credential-Dialog erscheint automatisch New-sqlAgentProxy -SqlInstance "SQL01" -CredentialName
"DOMAIN\SqlServiceAccount" -ProxyName "SSIS Proxy" # Credential direkt uebergeben - kein Dialog $cred = Get-
Credential "DOMAIN\SvcSSIS" New-sqlAgentProxy -SqlInstance "SQL01" -CredentialName "DOMAIN\SvcSSIS" -ProxyName
"SSIS Proxy" -WindowsCredential $cred
```

New-sqlAlwaysOnRepairJob

Creates a SQL Server Agent job that runs Repair-Job.ps1 (AutoRepair).

Parameter	Description
-SqlInstance	Target SQL instance (default: computer name).
-JobName	Name of the Agent job. Default: 'sqlAlwaysOnRepair'.
-Force	Overwrites an existing job.

Example:

```
New-sqmAlwaysOnRepairJob -SqlInstance "SQL01" -#> function New-sqmAlwaysOnRepairJob
{ [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'None')] param ( [Parameter(Mandatory = $false)]
[string]$SqlInstance = $env:COMPUTERNAME, [Parameter(Mandatory = $false)] [string]$JobName = 'sqmAlwaysOnRepair',
[Parameter(Mandatory = $false)] [switch]$Force ) process { # Setup output directory with proper permissions FIRST
SoutputPath = "C:\SystemWinSrvLog\MSSQL" if (-not (Test-Path $outputPath)) { New-Item -ItemType Directory -Path
SoutputPath -Force | Out-Null } # Grant permissions to SQL Agent service accounts @('NT SERVICE\MSSQLSERVER', 'NT
SERVICE\SQLSERVERAGENT') | ForEach-Object { $null = icacls $outputPath /grant "$ `:F" /T /C 2>&1 } # Check if job
exists $existingJob = Get-DbaAgentJob -SqlInstance $SqlInstance -Job $JobName -ErrorAction SilentlyContinue if
($existingJob -and -not $Force) { throw "Job '$JobName' existiert bereits. -Force zum Ueberschreiben." } if
($existingJob -and $Force) { Remove-DbaAgentJob -SqlInstance $SqlInstance -Job $JobName -Confirm:$false
-ErrorAction Stop } # Create job $job = New-DbaAgentJob -SqlInstance $SqlInstance -Job $JobName -Description
"AutoRepair: Repariert AlwaysOn-Datenbanken" -ErrorAction Stop # Create CmdExec job step (calls Repair-Job.ps1)
$psExePath = 'C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe' $repairScriptPath = 'C:\Program
Files\WindowsPowerShell\Modules\sqmSQLTool\jobs\Repair-Job.ps1' $command = "$psExePath -NoProfile
-ExecutionPolicy Bypass -File `"$repairScriptPath`" " $jobStep = New-DbaAgentJobStep -SqlInstance $SqlInstance
-Job $JobName -StepName "RunRepair" -Subsystem 'CmdExec' -Command $command -ErrorAction Stop # Add hourly
schedule $schedName = "sch $JobName" $schedSql = @" DECLARE @sid INT; WHILE EXISTS (SELECT 1 FROM
msdb.dbo.sysschedules WHERE name = N'$schedName') BEGIN SELECT TOP (1) @sid = schedule_id FROM
msdb.dbo.sysschedules WHERE name = N'$schedName'; EXEC msdb.dbo.sp delete schedule @schedule_id = @sid,
@force_delete = 1; END EXEC msdb.dbo.sp add schedule @schedule_name = N'$schedName', @enabled = 1, @freq_type =
4, @freq_interval = 1; EXEC msdb.dbo.sp attach schedule @job_name = N'$JobName', @schedule_name = N'$schedName';
"@ $null = Invoke-DbaQuery -SqlInstance $SqlInstance -Database msdb -Query $schedSql -ErrorAction Stop
[PSCustomObject]@{ SqlInstance = $SqlInstance JobName = $JobName Status = "Success" Message = "Job created"
Timestamp = Get-Date } } }
```

New-sqmAutoLoginSyncJob

Creates a SQL Agent job that runs Sync-Job.ps1 (AutoSync).

Parameter Description

-SqlInstance Target SQL instance (default: computer name).

-JobName Name of the Agent job. Default: 'sqmAutoLoginSync'.

-Force Overwrites an existing job.

Example:

```
New-sqmAutoLoginSyncJob -SqlInstance "SQL01" #> function New-sqmAutoLoginSyncJob
{ [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'None')] param ( [Parameter(Mandatory = $false)]
[string]$SqlInstance = $env:COMPUTERNAME, [Parameter(Mandatory = $false)] [string]$JobName = 'sqmAutoLoginSync',
[Parameter(Mandatory = $false)] [switch]$Force ) process { # Setup output directory with proper permissions FIRST
SoutputPath = "C:\System\WinSrvLog\MSSQL" if (-not (Test-Path $outputPath)) { New-Item -ItemType Directory -Path
SoutputPath -Force | Out-Null } # Grant permissions to SQL Agent service accounts @('NT SERVICE\MSSQLSERVER', 'NT
SERVICE\SQLSERVERAGENT') | ForEach-Object { $null = icacls $outputPath /grant "$ `:F" /T /C 2>&1 } # Check if job
exists $existingJob = Get-DbaAgentJob -SqlInstance $SqlInstance -Job $JobName -ErrorAction SilentlyContinue if
($existingJob -and -not $Force) { throw "Job '$JobName' existiert bereits. -Force zum Ueberschreiben." } if
($existingJob -and $Force) { Remove-DbaAgentJob -SqlInstance $SqlInstance -Job $JobName -Confirm:$false
-ErrorAction Stop } # Create job $job = New-DbaAgentJob -SqlInstance $SqlInstance -Job $JobName -Description
"AutoSync: Synchronisiert Logins ueber AlwaysOn-Replicas" -ErrorAction Stop # Create CmdExec job step (calls
Sync-Job.ps1) $psExePath = 'C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe' $syncScriptPath = 'C:
\Program Files\WindowsPowerShell\Modules\sqmSQLTool\jobs\Sync-Job.ps1' $command = "$psExePath -NoProfile
-ExecutionPolicy Bypass -File `"$syncScriptPath`"" $jobStep = New-DbaAgentJobStep -SqlInstance $SqlInstance -Job
$JobName -StepName "RunSync" -Subsystem 'CmdExec' -Command $command -ErrorAction Stop # Add daily schedule
(2:00 AM) $schedName = "sch $JobName" $schedSql = @" DECLARE @sid INT; WHILE EXISTS (SELECT 1 FROM
msdb.dbo.sysschedules WHERE name = N'$schedName') BEGIN SELECT TOP (1) @sid = schedule_id FROM
msdb.dbo.sysschedules WHERE name = N'$schedName'; EXEC msdb.dbo.sp delete schedule @schedule_id = @sid,
@force_delete = 1; END EXEC msdb.dbo.sp add schedule @schedule_name = N'$schedName', @enabled = 1, @freq_type =
4, @freq_interval = 1, @active_start_time = 020000; EXEC msdb.dbo.sp attach schedule @job_name = N'$JobName',
@schedule_name = N'$schedName'; "@ $null = Invoke-DbaQuery -SqlInstance $SqlInstance -Database msdb -Query
$schedSql -ErrorAction Stop [PSCustomObject]@{ SqlInstance = $SqlInstance JobName = $JobName Status = "Success"
Message = "Job created" Timestamp = Get-Date } } }
```

1. Always On & Availability Groups

Add-sqmDatabaseToAG

Adds one or more databases to an Always On availability group (AutoSeed).

Parameter	Description
-SqlInstance	Primary SQL instance (default: computer name).
-SqlCredential	Credentials.
-AvailabilityGroup	Name of the target availability group (mandatory).
-Database	Name or array of databases. Ignored when -All is set.
-All	Add all user databases that are not yet in an AG.
-EnableException	Allow exceptions to pass through.
-Confirm	Request confirmation.
-WhatIf	Test only (no changes).

Example:

```
Add-sqmDatabaseToAG -AvailabilityGroup "AG1" -Database "SalesDB"
```

Add-sqmDatabaseToDistributedAg

Adds a database to a Distributed AlwaysOn Availability Group.

Parameter	Description
-SqlInstance	Primary SQL Server instance. Default: current computer name.
-AvailabilityGroupName	Name of the Distributed AG.
-DatabaseName	Name of the database to add.
-SecondaryInstance	Secondary SQL Server instance where database will be restored.
-BackupPath	Path for full and log backups. Default: C:\Backups
-SqlCredential	Optional PSCredential for the connection.
-EnableException	Throw exceptions immediately.

Example:

```
Add-sqmDatabaseToDistributedAg -SqlInstance "SQL01" -AvailabilityGroupName "MyDAG" -DatabaseName "MyDb" -SecondaryInstance "DR-SQL01"
```

Compare-sqmAlwaysOnLogins

Vergleicht die Logins aller Replicas einer AlwaysOn Availability Group.

Parameter	Description
-SqlInstance	Einstiegs-Instanz der AG (Primary oder eine Secondary). Standard: aktueller Computer.
-AvailabilityGroupName	Name der Availability Group. Ohne Angabe wird die erste gefundene AG verwendet (bei meh...
-SqlCredential	Optionales PSCredential fuer alle Replicas.
-IncludeSystemLogins	Wenn gesetzt, werden auch Systemlogins (sa, ##MS_*, NT

Parameter	Description
	SERVICE*, NT AUTHORITY*, BUILT...
-Login	Nur diese Logins vergleichen (Wildcards erlaubt).
-ExcludeLogin	Diese Logins ausschliessen (Wildcards erlaubt).
-OnlyDifferences	Nur Logins mit Abweichung (Status Warning/Critical) ausgeben.
-OutputPath	Ausgabeverzeichnis fuer TXT/HTML. Standard: aus Modulkonfiguration.
-NoOpen	Unterdrueckt das automatische Oeffnen des Reports.
-FailOnDrift	Wenn gesetzt: bei Login-Drift (Status Warning oder Critical) wird ein Windows Event (So...
-ContinueOnError	Bei Fehler fortfahren.
-EnableException	Fehler sofort als Ausnahme ausloesen.

Example:

```
Compare-sqmAlwaysOnLogins -SqlInstance "SQL01"
```

Complete-sqmListenerMigration

Completes listener migration after cluster team recreates the listener resource.

Parameter	Description
-SqlInstance	SQL Server instance hosting the AG. Default: current computer name.
-AvailabilityGroupName	Name of the Availability Group.
-ListenerName	DNS name of the listener to be added (must match new cluster resource).
-OutputPath	Output directory for completion report. Default: C:

Parameter	Description
	\System\WinSrvLog\MSSQL
-EnableException	Throw exceptions immediately.
Example: <pre># STEP 1: DBA runs Invoke-sqmListenerMigrationPrep # STEP 2: AD team deletes/recreates listener role (15-30 min wait) # STEP 3: DBA runs this function Complete-sqmListenerMigration -SqlInstance "SQL02" -AvailabilityGroupName "ProdAG" -ListenerName "PROD-SQL-Listener"</pre>	
Export-sqmAlwaysOnConfiguration Exports the complete AlwaysOn AG configuration for one or more SQL Server instances.	
Parameter	Description
-SqlInstance	SQL Server instance(s). Pipeline-capable. Default: current computer name.
-SqlCredential	Optional PSCredential for the connection.
-OutputPath	Output directory for report files. Default: \$env:ProgramData\sqmSQLTool\Logs
-NoWarning	Suppress FI-TS ReadableSecondary warnings (Write-Warning is skipped). Note: Status wil...
-NoOpen	Do not automatically open the TXT report after creation.
-EnableException	Throw exceptions immediately (overrides ContinueOnError).
-Confirm	Request confirmation before writing files.
-WhatIf	Shows which files would be created without actually writing them.

Example:

```
Export-sqmAlwaysOnConfiguration -SqlInstance "SQL01" # Exports all AGs from SQL01, warns if ReadableSecondary != NO
```

Get-sqlAlwaysOnFailoverHistory

Ermittelt AlwaysOn-Failover-Ereignisse aus dem Windows Event Log.

Parameter	Description
<code>-ComputerName</code>	Zielcomputer. Standard: aktueller Computer. Mehrere Computer moeglich (Pipeline). Event...
<code>-SqlInstance</code>	SQL Server-Instanz fuer role_start_time-Ergaenzung. Optional. Wird nicht benoetigt wenn...
<code>-SqlCredential</code>	Optionales PSCredential fuer die SQL-Verbindung.
<code>-AvailabilityGroup</code>	Filter auf eine bestimmte AG. Leer = alle AGs.
<code>-Since</code>	Wie weit zurueck suchen. Standard: 30 Tage.
<code>-IncludeClusterLog</code>	WSFC Operational Log (Microsoft-Windows-FailoverClustering/Operational) zusaetzlich aus...
<code>-OutputPath</code>	Ausgabeverzeichnis. Standard: C:\System\WinSrvLog\MSSQL
<code>-ContinueOnError</code>	Bei Fehler auf einem Computer fortfahren.
<code>-EnableException</code>	Fehler als terminierende Ausnahmen ausloesen.

Example:

```
Get-sqlAlwaysOnFailoverHistory
```

Get-sqlAlwaysOnHealthReport

Creates a detailed health report for all Always On availability groups on an instance.

Parameter	Description
<code>-SqlInstance</code>	SQL Server instance(s). Pipeline-capable. Default: current computer name.
<code>-SqlCredential</code>	Optional PSCredential for the connection.

Parameter	Description
-MaxRedoQueueMB	Warning threshold for the redo queue in MB. Default: 100.
-MaxSendQueueMB	Warning threshold for the send queue in MB. Default: 50.
-OutputPath	Output directory for report files. Default: C:\System\WinSrvLog\MSSQL
-ContinueOnError	Continue on error for an instance (otherwise the error is thrown).
-EnableException	Throw exceptions immediately (overrides ContinueOnError).
-Confirm	Request confirmation before writing files.
-WhatIf	Shows which files would be created without actually writing them.

Example:

```
Get-sqmAgHealthReport
```

Get-sqmDistributedAgHealth

Creates a detailed health report for Distributed AlwaysOn Availability Groups.

Parameter	Description
-SqlInstance	SQL Server instance(s). Pipeline-capable. Default: current computer name.
-SqlCredential	Optional PSCredential for the connection.
-OutputPath	Output directory for report files. Default: C:\System\WinSrvLog\MSSQL
-ContinueOnError	Continue on error for an instance (otherwise the error is thrown).
-EnableException	Throw exceptions immediately (overrides ContinueOnError).

Example:

```
Get-sqmDistributedAgHealth -SqlInstance "SQL01"
```

Invoke-sqmDistributedFailover

Initiates failover of a Distributed AlwaysOn AG.

Parameter	Description
<code>-SqlInstance</code>	Primary SQL Server instance. Default: current computer name.
<code>-SqlCredential</code>	Optional PSCredential for the connection.
<code>-AvailabilityGroupName</code>	Name of the Distributed AG to failover. Required.
<code>-Force</code>	Skip confirmation dialog.
<code>-Rollback</code>	Rollback zum urspruenglichen Primary. Ueberspringt den Readiness-Check. Verwenden wenn ...
<code>-WhatIf</code>	Shows what would be done without actually performing the failover.
<code>-OutputPath</code>	Output directory for report files. Default: C:\System\WinSrvLog\MSSQL
<code>-EnableException</code>	Throw exceptions immediately.

Example:

```
Invoke-sqmDistributedFailover -SqlInstance "SQL01" -AvailabilityGroupName "MyDAG" -Force
```

Invoke-sqmFailover

Performs a controlled AlwaysOn AG failover with pre- and post-checks.

Parameter	Description
<code>-SqlInstance</code>	Current PRIMARY instance.
<code>-SqlCredential</code>	PSCredential for the connection.
<code>-AvailabilityGroup</code>	Name of the availability group.
<code>-TargetReplica</code>	Instance name of the target replica. If not specified, the first

Parameter	Description
	SYNCHRONIZED secondary...
<code>-MaxRedoQueueMB</code>	Maximum redo queue size in MB. Failover is aborted if exceeded. Default: 50 MB.
<code>-WaitAfterFailoverSeconds</code>	Wait time in seconds after the failover command before post-checks run. Default: 30 sec...
<code>-ContinueOnError</code>	Do not throw errors; return them in the result object instead.
<code>-EnableException</code>	Throw exceptions immediately.

Example:

```
Invoke-sqmFailover -SqlInstance "SQL01" -AvailabilityGroup "AG Prod" -WhatIf
```

`Invoke-sqmSqlAlwaysOnAutoseeding`

Enables Automatic Seeding on all replicas of an Always On Availability Group.

Parameter	Description
<code>-SqlInstance</code>	The target SQL Server instance (e.g. "localhost", "SQL01\INSTANCE"). If not specified,...
<code>-SqlCredential</code>	Alternative credentials (PSCredential). If not specified, Windows authentication is used.
<code>-AvailabilityGroup</code>	Name of the Availability Group(s). Ignored when -All is set.
<code>-All</code>	When set, all Availability Groups on the instance are processed.
<code>-EnableException</code>	Switch to propagate exceptions immediately (by default errors are logged as warnings).

Example:

```
# Uses the current computer name as default Invoke-sqmSqlAlwaysOnAutoseeding
```

Move-sqmAlwaysOnListener

Migrates an AG Listener from one Availability Group to another.

Parameter	Description
-SqlInstance	SQL Server instance hosting the SOURCE AG. Default: current computer name.
-SourceAgName	Name of the source AG (currently has the listener).
-TargetAgName	Name of the target AG (will receive the listener).
-TargetInstance	SQL Server instance hosting the target AG. Default: same as SourceInstance.
-ListenerName	Specific listener name to move (if multiple listeners exist). Optional.
-SqlCredential	Optional PSCredential for both instances.
-WhatIf	Shows what would be done without actually moving the listener.
-OutputPath	Output directory for detailed logs. Default: C:\System\WinSrvLog\MSSQL
-EnableException	Throw exceptions immediately.

Example:

```
# Move listener from Primary AG to Secondary AG (before failover) Move-sqmAgListener -SqlInstance "SQL01" -SourceAgName "ProductionAG" -TargetAgName "DrAG" -TargetInstance "DR-SQL01"
```

New-sqmDistributedAvailabilityGroup

Creates a new Distributed AlwaysOn Availability Group.

Parameter	Description
-PrimaryInstance	SQL Server instance hosting the PRIMARY Availability Group.
-PrimaryAgName	Name of the primary AG (the one that will remain primary).
-SecondaryInstance	SQL Server instance hosting the SECONDARY Availability Group.

Parameter	Description
-SecondaryAgName	Name of the secondary AG (the one that will be secondary in Distributed AG).
-SqlCredential	Optional PSCredential for both instances (same account required).
-EnableAutoSeed	Configure AutoSeed for the distributed relationship (recommended).
-SeedingMode	'Automatic' (default) = AutoSeed enabled 'Manual' = Manual backup/restore required for ...
-OutputPath	Output directory for detailed logs. Default: C:\System\WinSrvLog\MSSQL
-EnableException	Throw exceptions immediately.

Example:

```
New-sqmDistributedAvailabilityGroup ` -PrimaryInstance "SQL01" ` -PrimaryAgName "ProductionAG" ` -PrimaryFqdn "SQL01.domain.local" ` -SecondaryInstance "DR-SQL01" ` -SecondaryAgName "DrAG" ` -SecondaryFqdn "DR-SQL01.domain.local" ` -ServiceAccount "DOMAIN\SqlServiceAccount" ` -SeedingMode Automatic
```

Prepare-sqmListenerForMigration

Prepares an AG listener for cluster-level migration without downtime.

Parameter	Description
-SqlInstance	SQL Server instance hosting the AG. Default: current computer name.
-AvailabilityGroupName	Name of the Availability Group.
-ListenerName	DNS name of the listener to be removed (must exist). Optional if only one listener.
-SqlCredential	Optional PSCredential for the connection.
-OutputPath	Output directory for listener documentation. Default: C:

Parameter	Description
	\\System\\WinSrvLog\\MSSQL
-EnableException	Throw exceptions immediately.

Example:

```
# STEP 1: Prepare listener before AD team deletes it Invoke-sqlListenerMigrationPrep -SqlInstance "SQL01"
-AvailabilityGroupName "ProdAG" # STEP 2: Wait 15 minutes for DNS/application timeout # STEP 3: AD team deletes
old listener role and creates new one # STEP 4: You run Complete-sqlListenerMigration
```

Remove-sqlDatabaseFromAG

Removes one or more databases from their Always On Availability Group.

Parameter	Description
-SqlInstance	The primary SQL Server instance (the primary replica of the AG). Default: current comp...
-SqlCredential	Alternative credentials.
-Database	Name or array of user databases to remove from their AG. Ignored when -All is set.
-All	When set, all user databases that are members of an AG are removed.
-EnableException	Switch to propagate exceptions immediately.
-Confirm	Prompts for confirmation before critical actions (remove from AG, delete on secondaries).
-WhatIf	Shows what would happen without making any changes.

Example:

```
# Remove a single database from its AG Remove-sqlDatabaseFromAG -Database "SalesDB"
```

Repair-sqlAlwaysOnDatabases

Checks all AlwaysOn databases for problems and repairs them (Remove -> Cleanup -> Add).

Parameter	Description
<code>-SqlInstance</code>	Primary SQL instance (default: computer name).
<code>-SqlCredential</code>	Credentials.
<code>-Force</code>	Also repair databases that are considered healthy (e.g. to force a refresh).
<code>-EnableException</code>	Propagate exceptions immediately.
<code>-WhatIf</code>	Test only.

Example:

```
Automatically repairs all problematic AG databases. Repair-sqmAlwaysOnDatabases
```

`Sync-sqmLoginsToAlwaysOn`

Synchronizes logins from the primary replica to all secondary replicas in an AlwaysOn Availability Group.

Parameter	Description
<code>-SqlInstance</code>	The SQL Server instance hosting the primary replica. Default: <code>\$env:COMPUTERNAME</code>
<code>-AvailabilityGroupName</code>	Name of the Availability Group. If not specified, the first AG found on the instance is...
<code>-SqlCredential</code>	PSCredential for all replicas (source and destination).
<code>-SourceCredential</code>	PSCredential specifically for the primary replica (overrides <code>-SqlCredential</code> for source).
<code>-DestinationCredential</code>	PSCredential for the secondary replicas (overrides <code>-SqlCredential</code> for destinations).
<code>-Login</code>	Filters the copy operation to these login names (wildcards allowed). Without specificat...
<code>-ExcludeLogin</code>	Logins that should not be copied (wildcards allowed).

Parameter	Description
	Example: 'AppLogin_*', 'OldUser'.
-IncludeSystemLogins	When set, system logins are also copied. Default: \$false.
-AdjustAuthMode	When set, automatically adjust target replica authentication mode to match primary if n...
-RestartServiceIfRequired	When set, restart the SQL Server service on secondary replicas if auth mode was changed.
-DisablePolicy	Disable SQL Server policies on secondaries during the copy (default: \$true).
-SkipSecondaryServers	Comma-separated list of secondary instance names to skip (for maintenance). Example: 'S...

Example:

```
Sync-sqmLoginsToAlwaysOn -SqlInstance "SQL01" -AvailabilityGroupName "ProdAG" Syncs all logins from primary to all secondaries in ProdAG.
```

Test-sqmDistributedAgReadiness

Tests Distributed AlwaysOn AG readiness for failover.

Parameter	Description
-SqlInstance	Primary SQL Server instance. Default: current computer name.
-SqlCredential	Optional PSCredential for the connection.
-TargetInstance	Secondary SQL Server instance for network testing. Optional.
-OutputPath	Output directory for report files. Default: C:\System\WinSrvLog\MSSQL
-EnableException	Throw exceptions immediately.

Example:

```
Test-sqmDistributedAgReadiness -SqlInstance "SQL01" -TargetInstance "DR-SQL01"
```

18. SSIS Configuration

Test-sqmSSISPackageCompatibility

Validates SSIS package compatibility for SQL Server upgrades (2016 - 2025).

Parameter	Description
<code>-SqlInstance</code>	SQL Server instance to connect to (for SSISDB source). Omit to check only filesystem pa...
<code>-SqlCredential</code>	Optional PSCredential for SQL authentication.
<code>-FolderName</code>	Filter SSISDB packages to specific catalog folder(s). Example: 'MyFolder', 'Integration...
<code>-PackagePath</code>	Path to .dtsx files (filesystem source). Omit to check only SSISDB packages.
<code>-Recurse</code>	Recurse into subfolders when reading .dtsx files.
<code>-TargetVersion</code>	Target SQL Server version for compatibility check. Supported: 2016, 2017, 2019, 2022, 2...
<code>-OutputPath</code>	Directory for HTML/TXT/CSV reports. Default: <code>\$env:ProgramData\sqmSQLTool\SSISReports</code>
<code>-EnableException</code>	Throw exceptions instead of returning error status.

Example:

```
# Check deployed packages on target server Test-sqmSSISPackageCompatibility -SqlInstance "NewServer2025" -TargetVersion 2025
```

21. Analysis Services (SSAS)

Test-sqmSsasDirectoryPermissions

Checks and corrects NTFS permissions for SSAS directories (Data, Log, Temp, Backup).

Parameter	Description
<code>-InstanceName</code>	Name of the SSAS instance. Default: 'MSSQLSERVER' (default instance). For named instan...
<code>-ServiceAccount</code>	Optional: Name of the service account (e.g. 'NT SERVICE\MSSQLServerOLAPService'). If n...
<code>-WhatIf</code>	Shows which changes would be made without executing them.
<code>-Confirm</code>	Prompts for confirmation before each change.
<code>-EnableException</code>	Throws an exception immediately on errors (otherwise the error is logged).
<code>-ContinueOnError</code>	Continues checking the next directories even on errors.

Example:

```
Test-sqmSsasDirectoryPermissions Checks the directories of the default SSAS instance and corrects missing permissions.
```

2. Performance Analysis & Optimization

Get-sqmIndexFragmentation

Analyzes index fragmentation in one or more databases.

Parameter	Description
<code>-SqlInstance</code>	SQL Server instance (default: current computer name).
<code>-SqlCredential</code>	PSCredential for the connection.
<code>-Database</code>	Database name or wildcard pattern (e.g. 'Sales*'). Default: all user databases.
<code>-TableName</code>	Table name or wildcard pattern (e.g. 'Order*'). Default: all

Parameter	Description
	tables.
<code>-MinFragmentationPercent</code>	Show only indexes with fragmentation >= this value. Default: 5.
<code>-PageCountMin</code>	Show only indexes with at least this page count. Default: 0 (all indexes).
<code>-OutputPath</code>	Optional CSV export path.
<code>-EnableException</code>	Throw exceptions immediately.

Example:

```
Get-sqmIndexFragmentation -Database 'AdventureWorks' -MinFragmentationPercent 10
```

Get-sqmMissingIndexes

Retrieves missing index recommendations from the SQL Server DMV cache.

Parameter	Description
<code>-SqlInstance</code>	SQL Server instance (default: current computer name).
<code>-SqlCredential</code>	PSCredential for the connection.
<code>-Database</code>	Filter by database name(s). Wildcards allowed. Default: all user databases.
<code>-MinImpactScore</code>	Return only recommendations with impact score >= this value. Default: 10.
<code>-MinSeeks</code>	Return only recommendations with at least this number of seeks/scans. Default: 50.
<code>-Top</code>	Return at most this number of recommendations (sorted by impact score). Default: 50.

Parameter	Description
-OutputPath	If specified, a CSV file with the recommendations and CREATE statements is written to ...
-EnableException	Throw exceptions immediately.
Example: <pre>Get-sqmMissingIndexes -SqlInstance "SQL01"</pre>	
Get-sqmWaitStatistics	
Reads and analyzes SQL Server wait statistics from sys.dm_os_wait_stats.	
Parameter	Description
-SqlInstance	SQL Server instance. Default: local computer name.
-SqlCredential	PSCredential for the connection.
-TopN	Number of top wait types to return. Default: 25.
-IncludeIdle	Include idle waits (SLEEP_*, WAITFOR, etc.). Default: off.
-SnapshotBefore	PSCustomObject array of an earlier snapshot (output of -SaveSnapshot). If specified, on...
-SaveSnapshot	Returns a snapshot array that can later be used as SnapshotBefore.
-OutputPath	If specified, a CSV report is saved.
-EnableException	Throw exceptions immediately.

Example:

```
Get-sqmWaitStatistics -SqlInstance "SQL01" -TopN 20
```

Invoke-sqmPerfBaseline

Creates, compares or lists performance baselines (wait stats + perf counters).

Parameter	Description
-SqlInstance	SQL Server instance. Default: local computer name.
-SqlCredential	PSCredential for the connection.
-Action	Capture Compare List. Default: Capture.
-BaselineName	Label for the snapshot (used in the file name). Default: timestamp.
-BaselineA	Path or file name (without path) of the first baseline for comparison. Default: the sec...
-BaselineB	Path or file name of the second (newer) baseline. Default: the most recent file in Outp...
-OutputPath	Directory for baseline JSON files. Default: from module configuration + \PerfBaseline.
-EnableException	Throw exceptions immediately.

Example:

```
# Capture baseline Invoke-sqmPerfBaseline -SqlInstance "SQL01" -BaselineName "before patch"
```

Invoke-sqmQueryStore

Configures the Query Store, reads from it, detects issues and saves reports.

Parameter	Description
-SqlInstance	SQL Server instance. Default: current computer name.
-SqlCredential	PSCredential for the connection.
-Database	One or more databases. Ignored when -All is set.
-All	Process all accessible user databases.
-Configure	Configure Query Store (enable/set parameters).

Parameter	Description
-Query	Read top-N queries from the Query Store.
-Diagnose	Detect issues in the Query Store and return them as issues.
-OperationMode	Query Store operation mode. Values: READ_WRITE, READ_ONLY, OFF. Default: READ_WRITE.
-FlushIntervalSeconds	Frequency of writing to the Query Store (seconds). Default: 900.
-IntervalLengthMinutes	Length of a statistics interval (minutes). Default: 60.
-MaxStorageSizeMB	Maximum size of the Query Store (MB). Default: 1000.
-QueryCaptureMode	Capture mode. Values: ALL, AUTO, NONE. Default: AUTO.

Example:

```
# Report for all databases (Query + Diagnose) Invoke-sqmQueryStore -All
```

Test-sqmCostThreshold

Prueft ob CostThresholdForParallelism auf dem empfohlenen Wert liegt.

Parameter	Description
-SqlInstance	SQL Server-Instanz. Standard: lokaler Computernamen.
-SqlCredential	PSCredential fuer die SQL-Verbindung.
-MinRecommendedValue	Mindestwert fuer CostThresholdForParallelism. Standard: 50.
-EnableException	Ausnahmen sofort ausloesen statt Write-Error.

Example:

```
Test-sqmCostThreshold -SqlInstance "SQL01"
```

Test-sqmMaxDop

Prueft ob MAXDOP (Max Degree of Parallelism) korrekt konfiguriert ist.

Parameter	Description
-----------	-------------

-SqlInstance	SQL Server-Instanzname. Z.B. "MSSQLSERVER" oder "SERVER\INST01".
---------------------	--

Example:

```
Test-sqmMaxDop -SqlInstance 'MSSQLSERVER'
```

Test-sqmMaxMemory

Prueft ob SQL Server Max Server Memory korrekt konfiguriert ist.

Parameter	Description
-----------	-------------

-SqlInstance	SQL Server-Instanzname. Z.B. "MSSQLSERVER" oder "SERVER\INST01".
---------------------	--

-RecommendedPct	Empfohlener Prozentsatz des RAM fuer SQL Server. Standard: 90.
------------------------	--

Example:

```
Test-sqmMaxMemory -SqlInstance 'MSSQLSERVER'
```

Get-sqmTempDbRecommendation

Analyzes the TempDB configuration and provides optimization recommendations.

Parameter	Description
-----------	-------------

-SqlInstance	SQL Server instance (default: current computer name).
---------------------	---

-SqlCredential	PSCredential for the connection.
-----------------------	----------------------------------

-OutputPath	Optional CSV export path.
--------------------	---------------------------

-EnableException	Throw exceptions immediately.
-------------------------	-------------------------------

Example:

```

Get-sqmTempDbRecommendation -SqlInstance "SQL01" #> function Get-sqmTempDbRecommendation { [CmdletBinding()]
[OutputType([PSCustomObject])] param ( [Parameter(Mandatory = $false)] [string]$SqlInstance = $env:COMPUTERNAME,
[Parameter(Mandatory = $false)] [System.Management.Automation.PSCredential]$SqlCredential, [Parameter(Mandatory =
$false)] [string]$OutputPath, [Parameter(Mandatory = $false)] [switch]$EnableException ) begin { $functionName =
$MyInvocation.MyCommand.Name if (-not (Get-Module -ListAvailable -Name dbatools)) { throw "dbatools-Modul nicht
gefunden." } } process { try { $server = Connect-DbInstance -SqlInstance $SqlInstance -SqlCredential
$SqlCredential -ErrorAction Stop $tempdb = $server.Databases['tempdb'] $cpuCount = $server.Processors
$IdealFileCount = [Math]::Min($cpuCount, 8) $files = $tempdb.FileGroups[0].Files $fileCount = $files.Count
$fileSizeMB = $files | ForEach-Object { [math]::Round($_.Size / 1024, 2) } $fileGrowth = $files.ForEach('Growth')
| ForEach-Object { ($_.Size / 1024) } # in MB $growthTypes = $files.ForEach('GrowthType') $paths =
$files.ForEach('FileName') | ForEach-Object { Split-Path $_ -Parent } # Bewertung $status = 'OK' $messages =
[System.Collections.Generic.List[string]]::new() if ($fileCount -ne $IdealFileCount) { $status = 'Warning'
$messages.Add("Anzahl TempDB-Dateien: $fileCount (empfohlen $IdealFileCount).") } $sizeDifferences = ($fileSizeMB
| Select-Object -Unique).Count -gt 1 if ($sizeDifferences) { $status = 'Warning' $messages.Add("TempDB-Dateien
haben unterschiedliche Groessen: $($fileSizeMB -join ', ' MB.)" } $hasPercent = $growthTypes -contains 'Percent'
if ($hasPercent) { $status = 'Warning' $messages.Add("Autogrow in Prozent wird verwendet (MB empfohlen).") }
$hasLargeGrow = $fileGrowth -gt 1024 if ($hasLargeGrow) { $status = 'Warning' $messages.Add("Autogrow-
Schrittweite >1024 MB: $($fileGrowth -join ', ' MB.)" } $uniquePaths = $paths | Select-Object -Unique if
($uniquePaths.Count -eq 1) { $messages.Add("Alle TempDB-Dateien liegen auf demselben Laufwerk ($
($uniquePaths[0])) - fuer optimale Leistung separate Laufwerke empfehlenswert.") if ($status -eq 'OK') { $status
= 'Info' } } if ($messages.Count -eq 0) { $messages.Add("TempDB-Konfiguration ist optimal.") } $result =
[PSCustomObject]@{ SqlInstance = $SqlInstance Status = $status FileCount = $fileCount RecommendedCount =
$IdealFileCount FileSizesMB = $fileSizeMB GrowthMB = $fileGrowth Paths = $paths Recommendations = ($messages
-join ' ') } if ($OutputPath) { $result | Export-Csv -Path $OutputPath -NoTypeInformation -Encoding UTF8 -Force }
return $result } catch { Invoke-sqmLogging -Message $_.Exception.Message -FunctionName $functionName -Level
"ERROR" if ($EnableException) { throw } return $null } } }

```

Test-sqmTempDbFileCount

Prueft ob die Anzahl der TempDB-Datendateien der empfohlenen CPU-Anzahl entspricht.

Parameter	Description
-SqlInstance	SQL Server-Instanz. Standard: lokaler Computernamen.
-SqlCredential	PSCredential fuer die SQL-Verbindung.
-MaxFiles	Maximale empfohlene Dateianzahl. Standard: 8 (Microsoft-Empfehlung).
-EnableException	Ausnahmen sofort ausloesen statt Write-Error.

Example:

```
Test-sqmTempDbFileCount -SqlInstance "SQL01"
```

6. Certificates & TLS Security

Get-sqmCertificateReport

Creates a comprehensive report on SQL Server certificates and their expiration dates.

Parameter	Description
<code>-SqlInstance</code>	SQL Server instance(s). Pipeline-capable. Default: current computer name.
<code>-SqlCredential</code>	PSCredential for the connection.
<code>-WarningThresholdDays</code>	Certificates expiring in less than this number of days receive status 'Warning'. Default...
<code>-CriticalThresholdDays</code>	Certificates expiring in less than this number of days receive status 'Critical'. Defau...
<code>-IncludeUserDatabases</code>	Also include certificates in user databases. Default: \$false.
<code>-OutputPath</code>	Output directory for report files. Default: from module configuration.
<code>-ContinueOnError</code>	Continue on error for an instance instead of aborting.
<code>-EnableException</code>	Throw exceptions immediately.

Example:

```
Get-sqmCertificateReport
```

New-sqmSqlCertificate

Creates a new self-signed SQL Server certificate as a renewal of an existing one.

Parameter	Description
<code>-SqlInstance</code>	SQL Server instance (default: current computer name).
<code>-SqlCredential</code>	PSCredential for the connection.
<code>-CertificateName</code>	Name of the certificate to renew (exact name from sys.certificates).
<code>-Database</code>	Database where the certificate resides. Default: master.

Parameter	Description
-NewCertificateName	Name of the new certificate. Default: <OldName>_<Year> (e.g. AG_CERT_2027).
-ValidityYears	Validity period of the new certificate in years. Default: 5.
-BackupPath	Path for backing up the old certificate (.cer and .pvk). Default: from module configur...
-BackupEncryptionPassword	Password for encrypting the exported private key (.pvk). Required when the old certifi...
-RenameOldCertificate	Rename the old certificate after renewal (suffix _OLD_<date>). Default: \$true.
-BindEndpoint	Automatically bind the new certificate to the existing endpoint (AlwaysOn/Broker). Def...
-BindTde	Automatically activate the new certificate for TDE-encrypted databases. Default: \$fals...
-EnableException	Throw exceptions immediately.

Example:

```
# Simple renewal without automatic binding New-sqmSqlCertificate -SqlInstance "SQL01" -CertificateName "AG_CERT"
-BackupEncryptionPassword (Read-Host -AsSecureString)
```

Set-sqmSqlTlsCertificate

Binds a Windows certificate from the Machine store to SQL Server as the TLS certificate.

Parameter	Description
-SqlInstance	SQL Server instance name. For a default instance use the computer name or leave at defa...

Parameter	Description
-Thumbprint	Certificate thumbprint (hex string). Spaces are stripped automatically. Must match a ce...
-ForceEncryption	If specified, sets ForceEncryption = 1 in the SuperSocketNetLib registry key, requiring...
-Restart	If specified, restarts the SQL Server service automatically after the registry change. ...
-WhatIf	Shows what would be changed without making any modifications.
-Confirm	Prompts for confirmation before making changes.

Example:

```
Set-sqmSqlTlsCertificate -SqlInstance "SQL01" -Thumbprint "A1B2C3D4E5F6..." Binds the specified certificate to the default instance on SQL01. Service restart must be performed manually.
```

Set-sqmSrsHttpsCertificate

Bind a Windows certificate to SSRS or Power BI Report Server for HTTPS access.

Parameter	Description
-ComputerName	Target computer name or IP address. Default: localhost (\$env:COMPUTERNAME).
-Thumbprint	Mandatory. Certificate thumbprint (40 hex characters) from the LocalMachine\My store. ...
-Port	HTTPS port to bind. Default: 443.
-InstanceName	SSRS WMI instance name (e.g. "RS_SSRS", "RS_PBIRS"). Auto-detected when only one inst...
-IPAddress	IP address for the SSL binding. Default: "0.0.0.0" (all interfaces).
-RequireSSL	When specified, sets SecureConnectionLevel = 1 (HTTPS required). Default:

Parameter	Description
	SecureConne...
-Credential	PSCredential for the WinRM session (remote operation only).
-WhatIf	Shows what would happen without making any changes.
-Confirm	Prompts for confirmation before applying changes.

Example:

```
Set-sqmSrsHttpsCertificate -Thumbprint "A1B2C3D4E5F6A1B2C3D4E5F6A1B2C3D4E5F6A1B2" Binds the specified certificate to SSRS on the local machine on port 443. The SSRS instance is auto-detected.
```

Get-sqmTlsStatus

Audits TLS/SSL configuration and certificate status for all SQL Server instances on one or more computers.

Parameter	Description
-ComputerName	One or more computer names to audit. Default: current computer (\$env:COMPUTERNAME).
-Credential	Optional PSCredential used for Invoke-Command when auditing remote computers.
-OutputPath	Directory where the CSV and TXT report files are saved. Default: C:\System\WinSrvLog\M...
-WarnDaysBeforeExpiry	Number of days before certificate expiry that triggers a Warning status. Default: 60

Example:

```
Get-sqmTlsStatus Audits all SQL Server instances on the local computer and saves results to the default log folder.
```

15. Extended Events & Diagnostics

Get-sqmDeadlockReport

Reads and analyzes deadlock events from the System Health Extended Event session.

Parameter	Description
-SqlInstance	SQL Server instance (default: current computer name).
-SqlCredential	PSCredential for the connection.
-StartTime	Return only deadlocks from this point in time. Default: last 24 hours.
-EndTime	Return only deadlocks up to this point in time. Default: now.
-MaxDeadlocks	Maximum number of deadlocks returned (newest first). Default: 100.
-OutputPath	If specified, deadlock graphs are saved as .xdl files in this directory (format: Deadl...
-EnableException	Throw exceptions immediately instead of returning as errors.

Example:

```
Get-sqmDeadlockReport
```

Invoke-sqmExtendedEvents

Manages Extended Events sessions for performance analysis on SQL Server.

Parameter	Description
-SqlInstance	SQL Server instance. Default: current computer name.
-SqlCredential	PSCredential for the connection.
-SessionName	Name of the XEvent session. Default: 'sqmPerformance'.
-Template	Session template when creating. Values: SlowQueries, Blocking,

Parameter	Description
	Waits, Deadlocks, AllInO...
-SlowQueryThresholdMs	Minimum execution duration in milliseconds for SlowQueries capture. Default: 1000.
-WaitTypes	Comma-separated list of wait types for the Waits template. Default: LCK_M_X,LCK_M_S,LCK...
-TargetType	Target type for event storage: RingBuffer or File. Default: RingBuffer.
-TargetFilePath	Directory for XEL files (only for TargetType = File). Default: from module configuratio...
-MaxFileSizeMB	Maximum size of an XEL file (MB). Default: 100.
-MaxRolloverFiles	Number of XEL rollover files. Default: 5.
-RingBufferMaxMB	Maximum size of the ring buffer (MB). Default: 50.
-MaxEventsRead	Maximum number of events when reading. Default: 10000.

Example:

```
# Create AllInOne session and start immediately Invoke-sqmExtendedEvents -SqlInstance SQL01 -Create -Start
```

Invoke-sqmMonitoringKey

Gets or sets monitoring registry values for the sqmSQLTool on one or more computers.

Parameter	Description
-ComputerName	Target computer(s). Pipeline-capable. Default: current computer name.
-Operation	'Get' (default) reads the current values; 'Set' writes the specified values.

Parameter	Description
<code>-SQL</code>	SQL monitoring level: 'None', 'Standard', or 'Full'. Stored as DWORD (0/1/2) in the re...
<code>-SQLFreeSpaceVersion</code>	Free-space monitoring variant: 'Standard' (standalone) or 'Cluster' (AlwaysOn AG).
<code>-TSM</code>	TSM backup monitoring: 0 = inactive, 1 = active.
<code>-RegistryBase</code>	Registry hive path base under HKLM. Default: 'System'.
<code>-AutoDetectSQLFreeSpaceVersion</code>	When set (and -Operation Set), automatically detects whether the instance belongs to a...
<code>-Credential</code>	PSCredential for remote computer access.
<code>-ContinueOnError</code>	Continue with the next computer on error.
<code>-EnableException</code>	Throw exceptions immediately (overrides ContinueOnError).

Example:

```
Invoke-sqmMonitoringKey
```

9. Database Maintenance

Find-sqmDatabaseObject

Searches all (or selected) databases on an instance for an object name.

Parameter	Description
<code>-SqlInstance</code>	SQL Server instance (default: current computer name).
<code>-SqlCredential</code>	PSCredential for the connection.

Parameter	Description
-ObjectName	Name of the object to search for, or wildcard (e.g. '*customer*').
-ObjectType	Restrict to type: 'TABLE', 'VIEW', 'PROCEDURE', 'FUNCTION', 'TRIGGER', 'SYNONYM'. Mult...
-Database	Databases to search (wildcard, default: all user databases).
-IncludeSystem	Include system databases. Default: \$false.
-SearchDefinition	If \$true, the object text (definition) is also searched for <ObjectName> (slower).
-EnableException	Throw exceptions immediately.

Example:

```
Find-sqmDatabaseObject -SqlInstance "SQL01" -ObjectName "sp_GetOrders"
```

Invoke-sqmLogShrink

Shrinks the transaction log file (LDF) of one or more databases.

Parameter	Description
-SqlInstance	SQL Server instance (default: current computer name). For AG members, automatically re...
-SqlCredential	Optional PSCredential for the connection.
-Database	Target database name(s) (wildcards allowed). Without specification, all user databases...
-All	Processes all user databases (excl. system databases, online only). Also used implicit...
-ShrinkTargetPercent	Target size as a percentage of the current log size (1-99). Default: 10.

Parameter	Description
-MinTargetMB	Minimum target size in MB (default: 64 MB).
-ContinueOnError	Continue with the next database on error.
-EnableException	Throw exceptions immediately (overrides ContinueOnError).
-Confirm	Request confirmation before shrinking. Disabled by default.
-WhatIf	Shows what would happen without executing the shrink.

Example:

```
Invoke-sqlLogShrink -Database "MyDB" -ShrinkTargetPercent 20
```

Invoke-sqlSetDatabaseRecoveryMode

Changes the recovery mode of one or more user databases.

Parameter	Description
-SqlInstance	The target SQL Server instance (e.g. "localhost", "SQL01\INSTANCE"). If not specified,...
-SqlCredential	Alternative credentials (PSCredential). If not specified, Windows authentication is used.
-Database	Name or array of user databases whose recovery mode should be changed. Ignored when -A...
-All	When set, changes the recovery mode for all user databases.
-RecoveryMode	The desired recovery mode. Allowed values: Simple, Full, BulkLogged.
-EnableException	Switch to propagate exceptions immediately (by default errors are logged as warnings).
-Confirm	Prompts for confirmation before execution. Disabled by default. Passed

Parameter	Description
	through to Set-...
-WhatIf	Shows what would happen without actually making the change. Passed through to Set-Dbad...

Example:

```
# Set all user databases to Full (without prompting) Invoke-sqmSetDatabaseRecoveryMode -All -RecoveryMode Full
```

Invoke-sqmUpdateStatistics

Updates statistics in one or more databases.

Parameter	Description
-SqlInstance	SQL Server instance (default: current computer name).
-SqlCredential	PSCredential for the connection.
-Database	Database name or wildcard pattern.
-Table	Table name or wildcard pattern.
-Statistics	Statistic name or wildcard pattern.
-SamplePercent	Percentage of rows used for the update (0 = full scan). Default: 0.
-OnlyModified	Only update statistics that have changed since the last update. Default: \$true.
-Index	Also update statistics associated with an index. Default: \$true.
-WhatIf	Shows which statistics would be affected.
-EnableException	Throw exceptions immediately.

Example:

```
Invoke-sqmUpdateStatistics -Database 'SalesDB' -SamplePercent 10
```

Set-sqlDatabaseOwner

Sets the owner of one or more databases to a uniform login.

Parameter	Description
-SqlInstance	SQL Server instance(s). Pipeline-capable. Default: current computer name.
-SqlCredential	PSCredential for the connection.
-Database	Database name(s). Wildcards allowed (e.g. 'Prod*'). Default: all user databases.
-ExcludeDatabase	Databases to exclude. Wildcards allowed.
-OwnerLogin	Login to set as the new owner. Default: sa account (automatically determined via SID 0...
-IncludeSystemDatabases	Also include system databases (master, model, msdb). Default: \$false. tempdb is always...
-Force	Also process databases that already have the correct owner (forces re-assignment).
-OutputPath	Directory for the change log. Default: from module configuration.
-ContinueOnError	Continue on error for one instance. Default: \$false.
-EnableException	Throw exceptions immediately.

Example:

```
# Set sa account on all user databases Set-sqlDatabaseOwner -SqlInstance "SQL01"
```

22. Monitoring & Registry

Enable-sqmMonitoringAccess

Richtet einen Monitoring-Account auf allen SQL Server-Instanzen eines Computers ein.

Parameter	Description
<code>-ComputerName</code>	Zielcomputer. Standard: aktueller Computer.
<code>-MonitoringUser</code>	Windows-Login des Monitoring-Accounts (z.B. "DOMAIN\MonUser").
<code>-ServerRoleName</code>	Name der SQL Server-Rolle die angelegt wird. Standard: "MonitoringRole".
<code>-DatabaseRoleName</code>	Name der Datenbank-Rolle die in master und msdb angelegt wird. Standard: "MonitoringDbR..."
<code>-PolicyName</code>	Name einer SQL Server Policy die vor dem Setup deaktiviert und danach wieder aktiviert ...
<code>-OutputPath</code>	Ausgabeverzeichnis fuer das Log. Standard: C:\System\WinSrvLog\MSSQL
<code>-SqlCredential</code>	Optionales PSCredential fuer die SQL Server-Verbindung.
<code>-ContinueOnError</code>	Bei Fehler auf einer Instanz fortfahren statt abbrechen.
<code>-EnableException</code>	Fehler als terminierende Ausnahmen ausloesen.

Example:

```
Enable-sqmMonitoringAccess -MonitoringUser "CORP\SvcMonitoring"
```

Invoke-sqmMonitoringKey

Gets or sets monitoring registry values for the sqmSQLTool on one or more computers.

Parameter	Description
<code>-ComputerName</code>	Target computer(s). Pipeline-capable. Default: current

Parameter	Description
	computer name.
-Operation	'Get' (default) reads the current values; 'Set' writes the specified values.
-SQL	SQL monitoring level: 'None', 'Standard', or 'Full'. Stored as DWORD (0/1/2) in the re...
-SQLFreeSpaceVersion	Free-space monitoring variant: 'Standard' (standalone) or 'Cluster' (AlwaysOn AG).
-TSM	TSM backup monitoring: 0 = inactive, 1 = active.
-RegistryBase	Registry hive path base under HKLM. Default: 'System'.
-AutoDetectSQLFreeSpaceVersion	When set (and -Operation Set), automatically detects whether the instance belongs to a...
-Credential	PSCredential for remote computer access.
-ContinueOnError	Continue with the next computer on error.
-EnableException	Throw exceptions immediately (overrides ContinueOnError).

Example:

```
Invoke-sqmMonitoringKey
```

Invoke-sqmPatchAnalysis

Compares the installed SQL Server version with known CU/SP builds.

Parameter	Description
-SqlInstance	One or more SQL Server instances. Default: local computer name.

Parameter	Description
	Pipeline-capable.
<code>-SqlCredential</code>	PSCredential for the connection.
<code>-OutputPath</code>	If specified, a CSV report is saved.
<code>-EnableException</code>	Throw exceptions immediately.

Example:

```
Invoke-sqmPatchAnalysis -SqlInstance "SQL01"
```

20. Script Execution & Deployment

Invoke-sqmDeployScripts

Executes numbered SQL scripts from a directory sequentially against a SQL Server database.

Parameter	Description
<code>-SqlInstance</code>	SQL Server instance name (e.g. "SQLSERVER01" or "SQLSERVER01\INST1").
<code>-Database</code>	Target database name.
<code>-ScriptPath</code>	Directory that contains the numbered SQL script files.
<code>-LogPath</code>	Directory where the .log and .csv output files are written. Created if it does not exist.
<code>-JobNumber</code>	Optional job or order number. When provided it is embedded in the log filename: yyyyMMd...
<code>-QueryTimeout</code>	Timeout in seconds per script execution. Default: 30.
<code>-SkipBackup</code>	Skip the pre-deployment backup. Requires ShouldProcess

Parameter	Description
	confirmation (ConfirmImpact=High...
-NoWrapTransaction	Do not wrap all scripts in one outer transaction. Each script is responsible for its ow...
-SqlCredential	PSCredential for SQL Server authentication. When omitted Windows Authentication is used.

Example:

```
# Basic deploy with automatic backup Invoke-sqmDeployScripts -SqlInstance "SQLSERVER01" -Database "SalesDB" `
-ScriptPath "D:\Deploy\v2.3" -LogPath "D:\Logs\Deploy"
```

Invoke-sqmSignModule

Signs all PowerShell script files in a module directory using Set-AuthenticodeSignature.

Parameter	Description
-ModulePath	Path to the module root directory. All matching files are signed recursively. If omitte...
-CertificateThumbprint	Thumbprint of a certificate in Cert:\LocalMachine\My or Cert:\CurrentUser\My. If omitte...
-CertificatePath	Path to a .pfx file. Takes precedence over -CertificateThumbprint.
-CertificatePassword	SecureString password for the PFX file specified in -CertificatePath.
-TimestampServer	URL of the timestamp authority (TSA). Default: http://timestamp.digicert.com. On failur...
-IncludeExtensions	File extensions to sign. Default: @('.ps1', '.psm1', '.psd1').
-Force	Re-signs files that already carry a valid signature. Without -Force those files are ski...

Example:

```
# 1. Sign with a specific certificate from the store Invoke-sqmSignModule -ModulePath "C:\Dev\MyModule" `
-CertificateThumbprint "AB12CD34EF56..."
```

Copy-sqmToCentralPath

Copies one or more files to the configured CentralPath.